CSEC 101 // CYBERSECURITY LAB SERIES

03

LAB_03 > 03. Raven-CTF

Raven 1 — CTF Walkthrough

> WPScan → Hydra (SSH) → John the Ripper → `sudo python` (GTFOBins) → root

Επίπεδο: Αρχάριος | nmap | WPScan | Hydra | John | GTFOBins

```
root@lab:~# ./start_lab_03.sh
```

Εργαστηριακό Υλικό Κυβερνοασφάλειας – Εκπαιδευτική Χρήση

VulnHub / TryHackMe, Ethical Hacking

Αναλυτικές Σημειώσεις — Εργαστήριο 02: Raven 1 (VulnHub)

Το αρχείο αυτό είναι η **εμπλουτισμένη έκδοση** του `README.md` του εργαστηρίου. Το αρχικό κείμενο διατηρείται **αυτούσιο** και μετά από κάθε βήμα προστίθεται πλαίσιο *«Επεξήγηση»* με τη θεωρία πίσω από κάθε ενέργεια, την ανάλυση των εντολών και τα σημεία όπου οι φοιτητές δυσκολεύονται συχνότερα.

Η μεθοδολογία σε μία ματιά

Ένα μηχάνημα τύπου *boot-to-root* δεν επιλύεται με μεμονωμένες, ασύνδετες τεχνικές. Η επίλυσή του ακολουθεί έναν δομημένο μεθοδολογικό κύκλο, ο οποίος είναι κοινός σε κάθε επαγγελματικό έλεγχο διείσδυσης (penetration test) και αποτυπώνεται σε διεθνή πρότυπα όπως το **PTES** (Penetration Testing Execution Standard):

```
1. Reconnaissance   →  ποιος είναι ο στόχος; (IP, ports, services)
2. Enumeration      →  τι τρέχει και τι εκθέτει; (web, users, versions)
3. Exploitation     →  ποια ευπάθεια αξιοποιείται; (RCE, weak creds)
4. Post-exploit     →  τι βλέπω από μέσα; (files, configs, DB)
5. Privilege esc.   →  πώς από user γίνομαι root; (SUID, sudo, creds)
6. Reporting        →  τι έμαθα και πώς διορθώνεται; (flags, remediation)
```

Κάθε βήμα του παρόντος οδηγού αντιστοιχεί σε μία από τις φάσεις αυτές. Όταν η πορεία της επίλυσης φαίνεται να σταματά, ο αναλυτής οφείλει να επανέρχεται στο ερώτημα: *σε ποια φάση βρίσκομαι και ποια συγκεκριμένη πληροφορία μού λείπει για να μεταβώ στην επόμενη;* Η συστηματική αυτή προσέγγιση αντικαθιστά τις τυχαίες δοκιμές με τεκμηριωμένη μεθοδολογία.

Μαθησιακά αποτελέσματα: Με την ολοκλήρωση του εργαστηρίου, ο/η φοιτητής/φοιτήτρια θα είναι σε θέση να εντοπίζει ενεργούς κόμβους (hosts) σε ένα τοπικό δίκτυο, να ερμηνεύει την έξοδο του σαρωτή θυρών nmap, να ανακαλύπτει μη δημοσιευμένους καταλόγους και λογαριασμούς χρηστών, να κατανοεί για ποιον λόγο ένας αδύναμος κωδικός πρόσβασης υπονομεύει τη συνολική ασφάλεια ενός συστήματος, και να εξάγει διαπιστευτήρια (credentials) από αρχεία ρυθμίσεων όπως το `wp-config.php` καθώς και από πίνακες της βάσης δεδομένων MySQL.

Raven 1 CTF - Write-up (Επεξήγηση και Λύση)

Το **Capture the flag (CTF)** είναι μια δοκιμασία στον κυβερνοχώρο όπου ο παίκτης προσπαθεί να βρει κρυμμένα "σημάνσεις" (flags) μέσα σε ένα σύστημα (το λεγόμενο "box"). Αυτές οι προκλήσεις προσφέρουν μια εξαιρετική ευκαιρία για διασκέδαση, αυτοδίδακτη μάθηση και πρακτική εξάσκηση σε νέες δεξιότητες. Σήμερα, θα λύσουμε ένα CTF από την πλατφόρμα Vulnhub που ονομάζεται **Raven**.

Το **Raven 1** είναι ένα εξαιρετικό παράδειγμα ενός δομημένου CTF, με πολύ πρακτική προσέγγιση. Αυτό το απαιτητικό μηχάνημα έχει ιδιαίτερο ενδιαφέρον καθώς εμπλέκει τη χρήση πολλών διαφορετικών εργαλείων και τεχνικών.
Μπορείτε να κατεβάσετε το μηχάνημα από την επίσημη σελίδα: VulnHub - Raven: 1

> **Writeup αναφοράς:** Raven 1 CTF Write-up — Hariharan

Ο κεντρικός στόχος σε αυτό το CTF είναι να βρούμε **4 flags** τα οποία είναι πολύ καλά κρυμμένα μέσα στο σύστημα.

Βήμα 1: Αναγνώριση Δικτύου (Reconnaissance)

Επεξήγηση Βήματος 1 — γιατί το ARP και όχι το ping

Βασικές έννοιες. Ένα **τοπικό δίκτυο (LAN — Local Area Network)** είναι ένα δίκτυο περιορισμένης γεωγραφικής έκτασης στο οποίο οι συσκευές επικοινωνούν απευθείας μεταξύ τους. Κάθε συσκευή διαθέτει δύο διευθύνσεις: μια λογική **διεύθυνση IP** (επίπεδο δικτύου — OSI L3), που μπορεί να αλλάζει, και μια μοναδική φυσική **διεύθυνση MAC** (επίπεδο ζεύξης δεδομένων — OSI L2), ενσωματωμένη στην κάρτα δικτύου. Το πρωτόκολλο **ARP (Address Resolution Protocol)** είναι εκείνο που αντιστοιχίζει μια διεύθυνση IP στην αντίστοιχη MAC.

Για τον λόγο αυτό, στο πλαίσιο ενός LAN ο πιο αξιόπιστος έλεγχος ύπαρξης ενός κόμβου δεν είναι το πρωτόκολλο **ICMP** (που χρησιμοποιεί η εντολή `ping`), αλλά το ARP. Κάθε ενεργή κάρτα δικτύου υποχρεούται να απαντήσει στο ερώτημα «ποια διεύθυνση MAC αντιστοιχεί σε αυτή τη διεύθυνση IP;», διότι διαφορετικά η επικοινωνία μέσω Ethernet είναι αδύνατη. Αντίθετα, οι απαντήσεις ICMP μπορούν να φιλτραριστούν ή να απενεργοποιηθούν από ένα τείχος προστασίας (firewall), καθιστώντας το `ping` αναξιόπιστο ως μοναδικό κριτήριο.

Μέθοδος	Πρωτόκολλο	Μπλοκάρεται εύκολα;	Πότε χρησιμοποιείται
`ping` sweep	ICMP	**Ναι** (τείχη, ρυθμίσεις)	Γρήγορος έλεγχος
`netdiscover` / `arp-scan`	ARP (L2)	Όχι εντός LAN	Τοπικό δίκτυο εργαστηρίου
`nmap -sn`	ICMP + ARP + TCP	Μερικώς	Γενική σάρωση
`nmap -Pn`	TCP (χωρίς ping)	Όχι	Όταν το ICMP φιλτράρεται

Το εργαλείο `netdiscover`. Πρόκειται για εργαλείο ενεργητικής αναγνώρισης που αποστέλλει αιτήματα ARP σε ένα εύρος διευθύνσεων και καταγράφει τις απαντήσεις. Η τυπική σύνταξη είναι `netdiscover -r <εύρος>`:

- Η παράμετρος `-r` ορίζει το **εύρος σάρωσης** σε συμβολισμό **CIDR (Classless Inter-Domain Routing)**. Ο συμβολισμός αυτός εκφράζει πόσα από τα 32 δυαδικά ψηφία μιας διεύθυνσης IPv4 είναι σταθερά (το «πρόθεμα δικτύου»). Έτσι, το `/24` δεσμεύει τα πρώτα 24 ψηφία και αφήνει

ελεύθερα τα υπόλοιπα 8, δηλαδή $2^8 = 256$ διευθύνσεις, εκ των οποίων 254 είναι αξιοποιήσιμες για κόμβους (η πρώτη αντιστοιχεί στο δίκτυο και η τελευταία στη διεύθυνση εκπομπής — broadcast).
- Για κάθε κόμβο που απαντά, το εργαλείο εμφανίζει τη διεύθυνση IP, τη διεύθυνση MAC και τον **κατασκευαστή** της κάρτας δικτύου. Η ένδειξη «VMware, Inc.» αποτελεί ισχυρή ένδειξη ότι ο κόμβος είναι **εικονική μηχανή (Virtual Machine)**.

Η σημασία της διεύθυνσης MAC. Τα πρώτα τρία οκτάδες (bytes) μιας διεύθυνσης MAC συνιστούν το **OUI (Organizationally Unique Identifier)** και προσδιορίζουν τον κατασκευαστή. Σε έναν πραγματικό έλεγχο διείσδυσης, η εμφάνιση μιας MAC τύπου «VMware» ή «QEMU» μέσα σε ένα εταιρικό δίκτυο υποδηλώνει εικονική μηχανή — συχνά δοκιμαστικό ή ήδη παραβιασμένο σύστημα.

Συχνό σφάλμα. Ενδεχόμενη σάρωση του εύρους `/24` σε λανθασμένο υποδίκτυο (π.χ. σάρωση άλλου δικτύου ενώ το εργαστήριο βρίσκεται στο `10.0.2.0/24` του NAT του VirtualBox) δεν αποδίδει αποτελέσματα. Συνιστάται ο προηγούμενος έλεγχος της τοπικής διεύθυνσης με την εντολή `ip a`, ώστε να προσδιοριστεί το ορθό υποδίκτυο προς σάρωση.
Το πρώτο βήμα είναι να εντοπίσουμε τη διεύθυνση IP του μηχανήματος το οποίο τρέχει ως εικονική μηχανή (virtual machine) στο τοπικό μας δίκτυο. Για να το πετύχετε αυτό, μπορείτε να χρησιμοποιήσετε το εργαλείο `netdiscover` (π.χ. εκτελώντας `netdiscover -r <εύρος δικτύου>`) ή κάποιο άλλο εργαλείο σάρωσης δικτύου της επιλογής σας (όπως το `nmap -sn`).

Βήμα 2: Απαρίθμηση και Σάρωση Θυρών (Enumeration & Port Scanning)

Επεξήγηση Βήματος 2 — ερμηνεία της εξόδου του nmap

Τι είναι μια θύρα και μια υπηρεσία. Κάθε δικτυακή υπηρεσία «ακούει» σε μια αριθμημένη **θύρα (port)** — έναν λογικό δίαυλο επικοινωνίας στο επίπεδο μεταφοράς (OSI L4, πρωτόκολλα TCP/UDP). Η ίδια διεύθυνση IP μπορεί να εξυπηρετεί ταυτόχρονα πολλές υπηρεσίες, καθεμία σε διαφορετική θύρα (π.χ. SSH στη θύρα 22, HTTP στη θύρα 80). Ο εντοπισμός των ανοικτών θυρών αποκαλύπτει επομένως ποιες υπηρεσίες είναι διαθέσιμες και, κατ' επέκταση, ποια είναι η **επιφάνεια επίθεσης (attack surface)** του συστήματος.

Το **nmap (Network Mapper)** είναι το καθιερωμένο εργαλείο σάρωσης θυρών και αναγνώρισης υπηρεσιών. Η εντολή `nmap -sC -sV <IP>` συνδυάζει τρεις επιμέρους τεχνικές:

Παράμετρος	Τι κάνει	Τι προσθέτει στην πληροφορία
(κανένα)	TCP connect / SYN scan	Ποιες θύρες είναι **ανοιχτές**
`-sV`	Version detection	Ποια **υπηρεσία και έκδοση** (π.χ. Apache 2.4.10)
`-sC`	Default scripts	Banner, τίτλοι HTTP, ενδείξεις ευπαθειών

Τα τρία βασικά states μιας θύρας:

- **open** — κάποια υπηρεσία ακούει και δέχεται συνδέσεις.

- **closed** — δεν ακούει κανείς, αλλά το μηχάνημα απαντά (RST).
- **filtered** — δεν έχουμε απάντηση, την μπλοκάρει firewall ή την πετάει ο host.

Πώς μεταφράζεται το αποτέλεσμα σε σχέδιο επίθεσης:

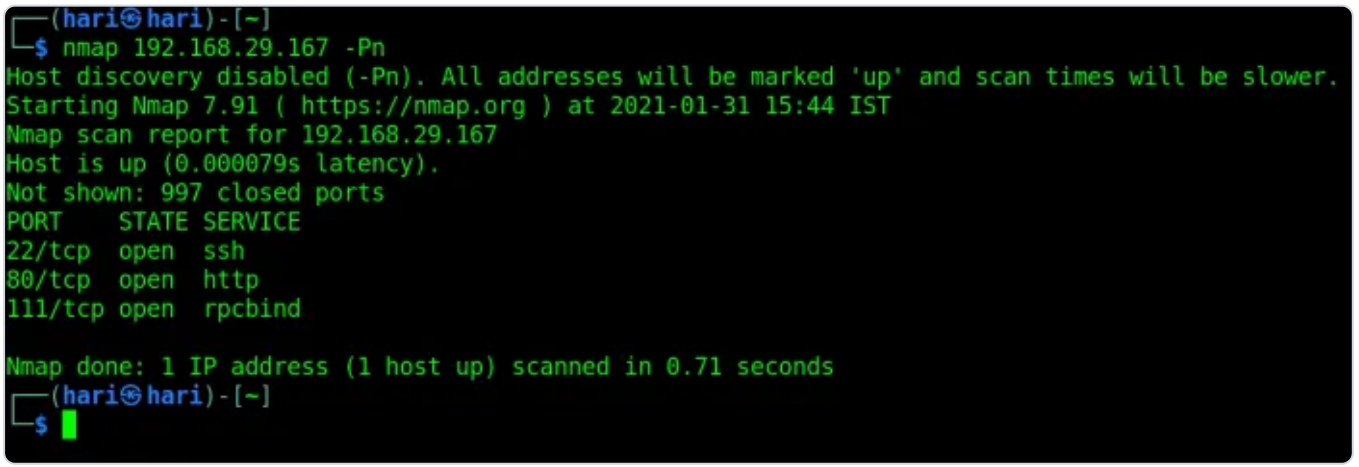
Θύρα	Υπηρεσία	Πρώτη σκέψη
22	SSH	Υπάρχουν γνωστοί χρήστες; Αδύναμοι κωδικοί;
80/443	HTTP/S	CMS, εκδόσεις, κρυφοί κατάλογοι, σχόλια σε κώδικα
111	rpcbind	NFS exports — συχνά «ανοιχτός» χωρίς έλεγχο
3306	MySQL	Credentials σε config αρχεία

Στο Raven 1 οι τρεις ανοιχτές θύρες (22, 80, 111) υπαγορεύουν μια σαφή στρατηγική: ο **ιστότοπος** αποτελεί την κύρια επιφάνεια επίθεσης, το **SSH** θα αποτελέσει το σημείο εισόδου μόλις εντοπιστεί έγκυρος κωδικός, ενώ η παρουσία του rpcbind υπενθυμίζει ότι καμία υπηρεσία δεν πρέπει να αγνοείται.

Χρήσιμες παραλλαγές:

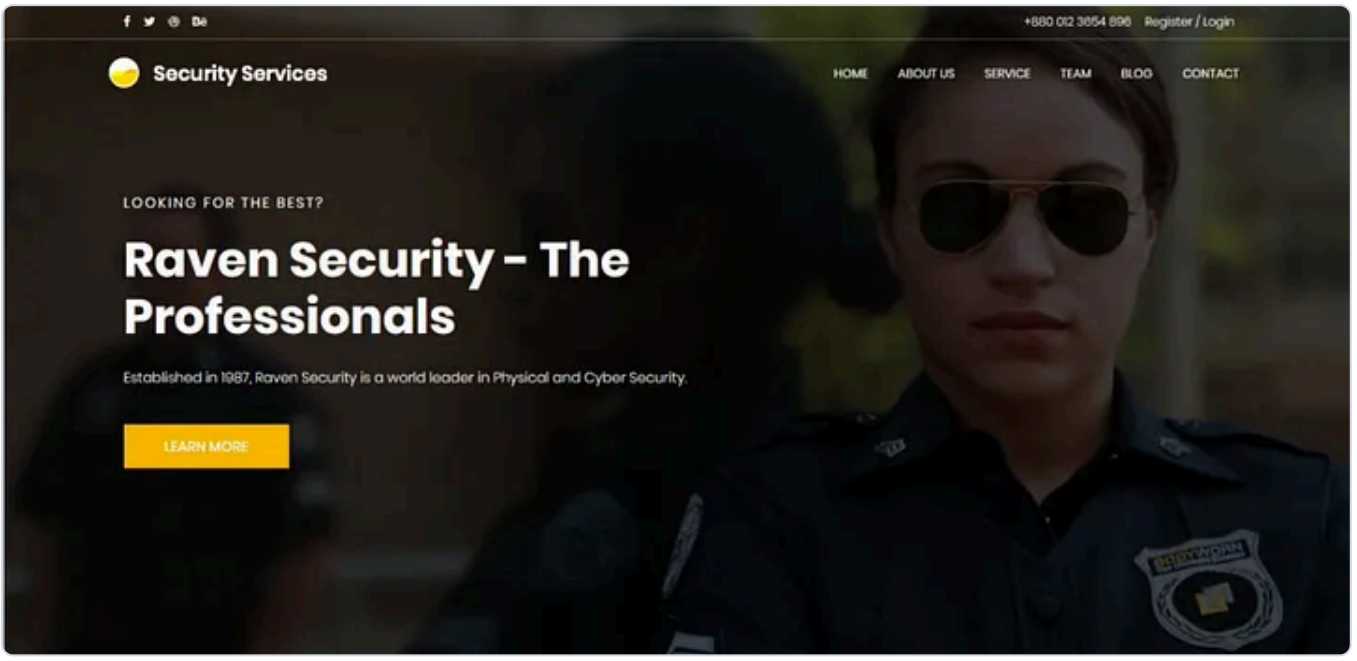
```
nmap -sV -p- 10.0.2.6            # και οι 65535 θύρες (αργό αλλά πλήρες)
nmap -sV -p 22,80,443 10.0.2.6   # στοχευμένα
nmap -A 10.0.2.6                 # -sV + -sC + OS detection + traceroute
```

Επισήμανση προσοχής: η παράμετρος `-sC` εκτελεί αυτοματοποιημένα σενάρια (scripts) τα οποία, σε μια ευάλωτη ή ασταθή υπηρεσία, ενδέχεται να προκαλέσουν δυσλειτουργία ή και κατάρρευσή της. Σε παραγωγικά περιβάλλοντα χρησιμοποιείται με φειδώ και αποκλειστικά κατόπιν ρητής εξουσιοδότησης.
Είναι πάντα εξαιρετικά σημαντικό να προχωράμε σε "απαρίθμηση" (enumeration) του μηχανήματος, παρά να κάνουμε τυφλές δοκιμές. Η κατανόηση του κάθε βήματος είναι κρίσιμη. Μια βασική σάρωση δικτύου (συνήθως με το **Nmap**) είναι απαραίτητη για κάθε CTF, προκειμένου να αποκαλυφθούν οι "ανοιχτές πόρτες" (open ports) και να εντοπιστούν πιθανές ευπάθειες χρησιμοποιώντας διάφορα scripts του εργαλείου (π.χ. με `nmap -sC -sV <IP>`).



Εικόνα 1 — Σάρωση Nmap: 22/tcp (ssh), 80/tcp (http) και 111/tcp (rpcbind) ανοιχτές.

Από τα αποτελέσματα της σάρωσης διαπιστώνουμε ότι υπάρχουν **3 ανοιχτά ports (θύρες)**. Από αυτές τις τρεις, οι υπηρεσίες **SSH** (συνήθως στο port 22) και **HTTP** (port 80) είναι οι πιο συνηθισμένες. Ξεκινάμε βάζοντας τη διεύθυνση IP του μηχανήματος στον Web Browser μας για να δούμε τι ακριβώς "σερβίρει" η σελίδα!



Εικόνα 2 — Η αρχική σελίδα «Raven Security» στην πύλη 80.

Βήμα 3: Ανάλυση της Ιστοσελίδας και Εύρεση του 1ου Flag

Επεξήγηση Βήματος 3 — τι ψάχνουμε μέσα σε μια σελίδα

Το πρώτο flag σε CTF κρύβεται συνήθως σε σημείο που **δεν φαίνεται** αλλά υπάρχει στο HTML. Τα κλασικά σημεία ελέγχου:

Σημείο	Πώς το βλέπετε	Τι μπορεί να κρύβει
HTML comments	`Ctrl+U` ή DevTools (F12)	Flags, ονόματα χρηστών, paths
Meta tags	`<meta name="generator">`	CMS και έκδοση (π.χ. WordPress 5.x)
`robots.txt`	`http://host/robots.txt`	Καταλόγους που «δεν θέλουν» να δουν
`sitemap.xml`	`http://host/sitemap.xml`	Δομή του site
JavaScript αρχεία	DevTools → Sources	API endpoints, κλειδιά, σχόλια
HTTP headers	`curl -I http://host`	Server, τεχνολογίες

Τα εργαλεία προγραμματιστή (Developer Tools). Κάθε σύγχρονος φυλλομετρητής (browser) διαθέτει ενσωματωμένα εργαλεία ανάπτυξης, προσβάσιμα με το πλήκτρο `F12` ή με τον συνδυασμό `Ctrl+U` (προβολή πηγαίου κώδικα). Μέσω αυτών ο αναλυτής εξετάζει τον πηγαίο κώδικα **HTML**

που ο διακομιστής απέστειλε στον φυλλομετρητή — δηλαδή ό,τι αποστέλλεται προς τον πελάτη (client-side), ανεξαρτήτως του αν εμφανίζεται οπτικά στη σελίδα.

Γιατί τα σχόλια HTML αποτελούν συχνό εύρημα. Τα σχόλια της μορφής `<!-- ... -->` δεν αποδίδονται οπτικά, παραμένουν όμως πλήρως ορατά σε όποιον εξετάσει τον πηγαίο κώδικα. Οι προγραμματιστές εισάγουν τέτοια σχόλια κατά την ανάπτυξη και συχνά παραλείπουν να τα αφαιρέσουν πριν τη δημοσίευση. Το φαινόμενο αυτό συνιστά το απλούστερο παράδειγμα **διαρροής πληροφορίας (information disclosure)**: αποκάλυψη ευαίσθητων στοιχείων χωρίς να απαιτείται καν η εκμετάλλευση κάποιας ευπάθειας λογισμικού.

Ταυτόχρονη πληροφορία που πρέπει να κρατήσετε: από τη σελίδα της Raven Security μαθαίνετε ότι πρόκειται για εταιρικό site με WordPress — άρα το επόμενο βήμα (directory fuzzing) έχει συγκεκριμένο στόχο: το `/wordpress/`.

Έλεγχος από τερματικό (χρήσιμο όταν δεν έχετε browser):

```
curl -s http://10.0.2.6/ | grep -i "flag\|comment"
```

Η ιστοσελίδα έχει πολλούς υπερσυνδέσμους (hyperlinks). Το αμέσως επόμενο πράγμα που πρέπει να ελέγξει κάθε επαγγελματίας ασφαλείας είναι τα **Εργαλεία Προγραμματιστών (Developer Tools)** του Browser (πατώντας F12) και να κάνει "Επιθεώρηση" (Inspect) στον πηγαίο κώδικα (HTML) της σελίδας. Πολλές φορές, τα CTF κρύβουν χρήσιμα στοιχεία σε μορφή σχολίων HTML (`<!-- σχόλιο -->`).



Εικόνα 3 — Το `<!--flag{...}-->` κρυμμένο στα σχόλια του HTML.

Και πράγματι, το **Πρώτο Flag (Flag 1)** είναι πολύ καλά κρυμμένο μέσα στον κώδικα στα Developer Tools!

Βήμα 4: Brute-Force στους Καταλόγους (Directory Fuzzing)

Επεξήγηση Βήματος 4 — η λογική του directory fuzzing

Ορισμός. Το **directory fuzzing** (ή directory brute-forcing) είναι η αυτοματοποιημένη δοκιμή πλήθους πιθανών ονομάτων καταλόγων και αρχείων σε έναν διακομιστή ιστού, με σκοπό τον εντοπισμό πόρων που δεν είναι συνδεδεμένοι (linked) από τις ορατές σελίδες. Η μέθοδος δεν βασίζεται σε

τυχαίες δοκιμές, αλλά σε **λεξικά (wordlists)** που περιέχουν χιλιάδες συνήθη ονόματα (π.χ. `common.txt`, `directory-list-2.3-medium.txt`). Εάν ο διαχειριστής έχει αφήσει προσβάσιμους καταλόγους όπως `/backup/`, `/vendor/` ή `/wordpress/` χωρίς δημόσιο σύνδεσμο προς αυτούς, το εργαλείο τους εντοπίζει μέσω των αποκρίσεων του διακομιστή.

Πώς ξεχωρίζετε «υπάρχει» από «δεν υπάρχει» — κωδικοί HTTP:

Κωδικός	Σημασία	Τι σημαίνει για εμάς
200	OK	Η σελίδα/κατάλογος υπάρχει και είναι προσβάσιμη
301/302	Redirect	Υπάρχει, αλλά σε άλλο path (π.χ. `/wordpress` → `/wordpress/`)
403	Forbidden	**Υπάρχει** αλλά απαγορεύεται η πρόσβαση — πληροφορία από μόνη της
401	Unauthorized	Απαιτεί login
404	Not found	Δεν υπάρχει

Γιατί το 403 είναι χρήσιμο: επιβεβαιώνει ύπαρξη αρχείου/καταλόγου. Σε πραγματικό audit, ένα `403` σε `/admin/` λέει «υπάρχει πίνακας διαχείρισης» — και ίσως τον προσεγγίσετε με άλλο τρόπο (παράκαμψη, γνωστό CVE).

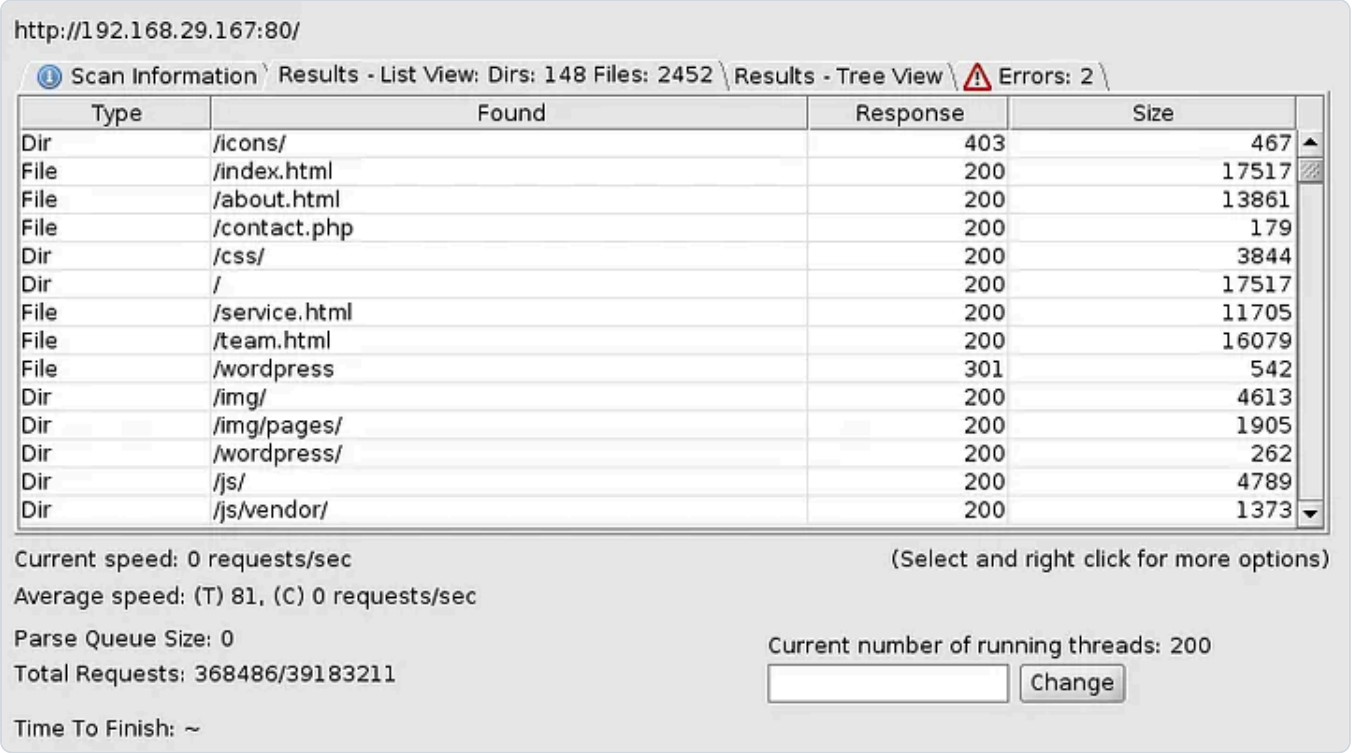
Σύγκριση εργαλείων:

Εργαλείο	Πλεονέκτημα	Μειονέκτημα
DirBuster	Γραφικό, εύκολο για αρχάριους	Αργό, βαρύ
gobuster	Γρήγορο, CLI, scripts	Θέλει σωστό wordlist
ffuf	Πολύ γρήγορο, φίλτρα μεγέθους	Καμπύλη εκμάθησης

Παράδειγμα με gobuster:

```
gobuster dir -u http://10.0.2.6/ -w /usr/share/wordlists/dirb/common.txt -x php,txt
```

Στο Raven 1 το αποτέλεσμα `301 → /wordpress/` είναι το κρίσιμο εύρημα: μας λέει ότι υπάρχει CMS, άρα υπάρχει διαχείριση χρηστών και plugins — δηλαδή επιφάνεια επίθεσης.
Το επόμενο λογικό βήμα, μετά τον έλεγχο της αρχικής σελίδας, είναι ο εντοπισμός μη δημοσιευμένων καταλόγων (directories) στον διακομιστή. Για τον σκοπό αυτό χρησιμοποιείται το **DirBuster** (ή αντίστοιχα εργαλεία όπως τα Gobuster/Dirb). Πρόκειται για ιδιαίτερα διαδεδομένη τεχνική, η οποία επιτρέπει τον εντοπισμό μη προσβάσιμων μέσω συνδέσμων διαδρομών (paths) χωρίς τυφλές δοκιμές.



Εικόνα 4 — DirBuster: εντοπίστηκε ο κατάλογος `/wordpress/` (HTTP 301).

Τα αποτελέσματα του εργαλείου μάς αποκαλύπτουν την ύπαρξη ενός καταλόγου ονόματι **WordPress**.

Βήμα 5: Αξιολόγηση Ευπαθειών με το WPScan

Επεξήγηση Βήματος 5 — η λειτουργία του WPScan

Τι είναι ένα CMS. Το **WordPress** είναι ένα **Σύστημα Διαχείρισης Περιεχομένου (CMS — Content Management System)**: μια έτοιμη πλατφόρμα λογισμικού που επιτρέπει τη δημιουργία και διαχείριση ιστοσελίδων χωρίς εξειδικευμένη ανάπτυξη κώδικα, με επεκτάσιμη λειτουργικότητα μέσω **προσθέτων (plugins)** και **θεμάτων (themes)**. Ακριβώς λόγω της ευρείας διάδοσής του, το WordPress αποτελεί συχνό στόχο, καθώς μια γνωστή ευπάθεια σε ένα δημοφιλές plugin επηρεάζει ταυτόχρονα πλήθος ιστοτόπων.

Τι είναι το WPScan. Πρόκειται για σαρωτή ειδικά σχεδιασμένο για WordPress. Δεν παραβιάζει τον ιστότοπο, **συλλέγει** πληροφορίες (παθητική και ενεργητική απαρίθμηση):

- **Έκδοση** WordPress (από meta tags, readme, feeds).
- **Plugins/themes** και τις εκδόσεις τους → σύγκριση με γνωστές ευπάθειες (WPScan API).
- **Χρήστες** — μέσω του `?author=1` enumeration ή της REST API.

Ανάλυση της εντολής της άσκησης:

```
wpscan --url http://10.0.2.6/wordpress -e u
```

Τμήμα	Σημασία
`--url`	Ο στόχος (το path του WordPress, όχι απαραίτητα η ρίζα)
`-e u`	*enumerate users* — λίστα χρηστών
`-e vp`	ευάλωτα plugins
`-e vt`	ευάλωτα themes
`--api-token`	για πλήρη βάση ευπαθειών (δωρεάν token)

Γιατί η απαρίθμηση χρηστών είναι κρίσιμη: το WordPress επιτρέπει σύνδεση με **username + password**. Αν γνωρίζετε το username (`michael`, `steven`), το brute-force γίνεται εφικτό — δεν χρειάζεται να μαντέψετε και τα δύο.

Πώς αμύνεται ένα site: απενεργοποίηση author enumeration, 2FA, όριο προσπαθειών (rate limiting), ισχυροί κωδικοί, απόκρυψη έκδοσης. Κάθε ένα από αυτά θα είχε σταματήσει την πορεία αυτού του CTF.

Σημείωση δεοντολογίας: το WPScan στέλνει εκατοντάδες αιτήματα, σε ξένο site αυτό είναι επίθεση. Εδώ το κάνουμε σε τοπική VM.
Το σύστημα διαχείρισης περιεχομένου WordPress έχει ένα αρκετά κακό ιστορικό όσον αφορά τις ευπάθειες. Ξεκινώντας μια σάρωση με το εξειδικευμένο εργαλείο **WPScan**, μπορούμε να ανακαλύψουμε την ακριβή έκδοση του WordPress, τυχόν ευπαθή πρόσθετα (plugins), αλλά και να απαριθμήσουμε τους χρήστες.

Η σύνταξη της εντολής WPScan ενδέχεται να φανεί αρχικά περίπλοκη (π.χ. `wpscan --url http://<IP>/wordpress/ -e u`). Σε κάθε περίπτωση δυσκολίας, η ενσωματωμένη τεκμηρίωση του εργαλείου, μέσω των εντολών `man wpscan` ή `wpscan --help`, παρέχει πλήρη επεξήγηση των διαθέσιμων παραμέτρων.

```
┌──(hari㉿hari)-[~]
└─$ wpscan --url http://192.168.29.167/wordpress --wp-content-dir -ep -et -eu
```

Εικόνα 5 — Εκτέλεση του WPScan με απαρίθμηση χρηστών (`-eu`) και plugins/themes (`-ep -et`).

Η σάρωση μπορεί να πάρει μερικά λεπτά για να ολοκληρωθεί.

Μετά την ολοκλήρωση της σάρωσης, η βάση δεδομένων του εργαλείου μάς αποκαλύπτει τα **ονόματα των χρηστών (usernames)**. Ίσως μπορούμε να χρησιμοποιήσουμε αυτά τα usernames για να συνδεθούμε στο μηχάνημα μέσω SSH; Ας δούμε!

```
[+] steven
 | Found By: Author Id Brute Forcing - Author Pattern (Aggressive Detection)
 | Confirmed By: Login Error Messages (Aggressive Detection)

[+] michael
 | Found By: Author Id Brute Forcing - Author Pattern (Aggressive Detection)
 | Confirmed By: Login Error Messages (Aggressive Detection)
```

Εικόνα 6 — WPScan: εντοπίστηκαν οι χρήστες `steven` και `michael`.

Βήμα 6: Πρόσβαση στο Σύστημα και Εύρεση 2ου Flag

Επεξήγηση Βήματος 6 — γιατί ο «προφανής» κωδικός είναι ρεαλιστικό σενάριο

Ορισμός — επίθεση ωμής βίας (brute-force). Πρόκειται για τη συστηματική, αυτοματοποιημένη δοκιμή πολλών υποψήφιων κωδικών πρόσβασης έως ότου εντοπιστεί ο ορθός. Όταν οι δοκιμαζόμενοι κωδικοί προέρχονται από προκαθορισμένη λίστα συνήθων κωδικών, η τεχνική ονομάζεται **επίθεση λεξικού (dictionary attack)**.

Ο συνδυασμός `michael:michael` (όνομα χρήστη ταυτόσημο με τον κωδικό) ενδέχεται να φαίνεται εξωπραγματικός, αποτελεί όμως απολύτως ρεαλιστικό σενάριο. Οι επιθέσεις **credential stuffing** (δοκιμή διαπιστευτηρίων που έχουν διαρρεύσει από άλλες υπηρεσίες) και **password spraying** (δοκιμή λίγων πολύ συνήθων κωδικών σε πολλούς λογαριασμούς) βασίζονται ακριβώς σε τέτοιες επισφαλείς συνήθειες των χρηστών: όνομα χρήστη ως κωδικός, `Password1`, εποχή συνοδευόμενη από `!` κ.ο.κ. Στατιστικά, ένα μικρό λεξικό προσεκτικά επιλεγμένων κωδικών —και όχι ολόκληρο το `rockyou.txt` — επαρκεί για σημαντικό ποσοστό επιτυχίας.

Το εργαλείο `hydra` και η ταχύτητα της επίθεσης. Το `hydra` είναι εργαλείο αυτοματοποιημένης δοκιμής διαπιστευτηρίων σε πλήθος πρωτοκόλλων, μεταξύ των οποίων και το SSH. Η επίθεση ωμής βίας στο SSH είναι εγγενώς βραδεία, διότι κάθε μεμονωμένη δοκιμή απαιτεί πλήρη **χειραψία (handshake)** TCP και, εν συνεχεία, κρυπτογραφική χειραψία SSH. Για τον λόγο αυτό το `hydra` εκτελεί πολλές δοκιμές παράλληλα (παράμετρος `-t`). Η ίδια ακριβώς ιδιότητα, ωστόσο, καθιστά την επίθεση ευδιάκριτη: οι διακομιστές αντιμετωπίζουν τέτοιες προσπάθειες με μηχανισμούς όπως το `fail2ban` (προσωρινός αποκλεισμός διεύθυνσης IP έπειτα από επαναλαμβανόμενες αποτυχίες) ή με περιορισμό ρυθμού συνδέσεων.

```
hydra -l michael -P /usr/share/wordlists/rockyou.txt ssh://10.0.2.6
hydra -L users.txt -P pass.txt -t 4 ssh://10.0.2.6     # λίστες και για τα δύο
```

Τι κάνουμε αφού συνδεθούμε — η λογική του enumeration ως `michael`:

1. **Ποιος είμαι;** `id`, `whoami`, `sudo -l` (τι μπορώ να τρέξω ως root;)
2. **Πού είμαι;** `pwd`, `ls -la`, `cat /etc/passwd` (άλλοι χρήστες)
3. **Τι μπορώ να διαβάσω;** `find / -name "flag*" 2>/dev/null`, `ls /var/www`
4. **Τι υπηρεσίες τρέχουν;** `ps aux`, `ss -tuln`

Γιατί το `sudo -l` είναι το πρώτο που τρέχουμε: αν ο χρήστης έχει δικαίωμα `sudo` σε κάποια εντολή (π.χ. `less`, `find`, `vim`), η κλιμάκωση είναι σχεδόν σίγουρη. Στο Raven 1 η διαδρομή είναι διαφορετική (επαναχρησιμοποίηση κωδικού), αλλά η συνήθεια μένει.

Το flag 2 βρίσκεται στο `/var/www/flag2.txt`: επιβεβαιώνει ότι ο web root είναι προσβάσιμος από τον χρήστη και ότι τα «μυστικά» του site δεν προστατεύονται.
Εδώ υπάρχει ένα πρόβλημα. Ο κωδικός πρόσβασης (password) για το username που βρήκαμε

(χρήστης `michael`) μάς είναι άγνωστος. Σε αυτές τις περιπτώσεις, είναι πολύ καλή πρακτική να δοκιμάζουμε πάντα τους πιο "κοινούς" ή προφανείς κωδικούς πριν ξεκινήσουμε χρονοβόρες επιθέσεις brute-force.

Συγκεκριμένα, ο χρήστης εκτελεί την εντολή `hydra -l michael -P /usr/share/wordlists/rockyou.txt ssh://10.0.2.6`, η οποία στοχεύει τον λογαριασμό με όνομα `michael` στην IP διεύθυνση `10.0.2.6`, αξιοποιώντας το γνωστό λεξικό κωδικών `rockyou.txt` για τη μαζική και αυτοματοποιημένη δοκιμή χιλιάδων πιθανών κωδικών πρόσβασης. Όπως φαίνεται στα αποτελέσματα της εκτέλεσης στο κάτω μέρος του τερματικού, η επίθεση ολοκληρώνεται με επιτυχία, καθώς το εργαλείο εντοπίζει έναν έγκυρο κωδικό, αποκαλύπτοντας ότι το password για τον χρήστη `michael` είναι επίσης η λέξη «michael».

```
┌──(rafiq㉿Hacker)-[~/Desktop/vulnhub/raven-1]
└─$ hydra -l michael  -P /usr/share/wordlists/rockyou.txt ssh://10.0.2.6
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service
binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2025-04-13 10:53:08
[WARNING] Many SSH configurations limit the number of parallel tasks, it is recommended to reduce the ta
[WARNING] Restorefile (you have 10 seconds to abort... (use option -I to skip waiting)) from a previous
e
[DATA] max 16 tasks per 1 server, overall 16 tasks, 14344399 login tries (l:1/p:14344399), ~896525 tries
[DATA] attacking ssh://10.0.2.6:22/
[22][ssh] host: 10.0.2.6   login: michael   password: michael
1 of 1 target successfully completed, 1 valid password found
[WARNING] Writing restore file because 2 final worker threads did not complete until end.
[ERROR] 2 targets did not resolve or could not be connected
[ERROR] 0 target did not complete
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2025-04-13 10:53:26
```

Εικόνα 7 — `hydra -l michael -P rockyou.txt ssh://…`: επιτυχής brute-force, `login: michael password: michael`.

Εδώ, κάνοντας τη δοκιμή, διαπιστώνουμε ότι το username και το password ήταν τα ίδια: **“michael”**. Συνδεόμαστε λοιπόν με επιτυχία μέσω SSH (`ssh michael@<IP>`).

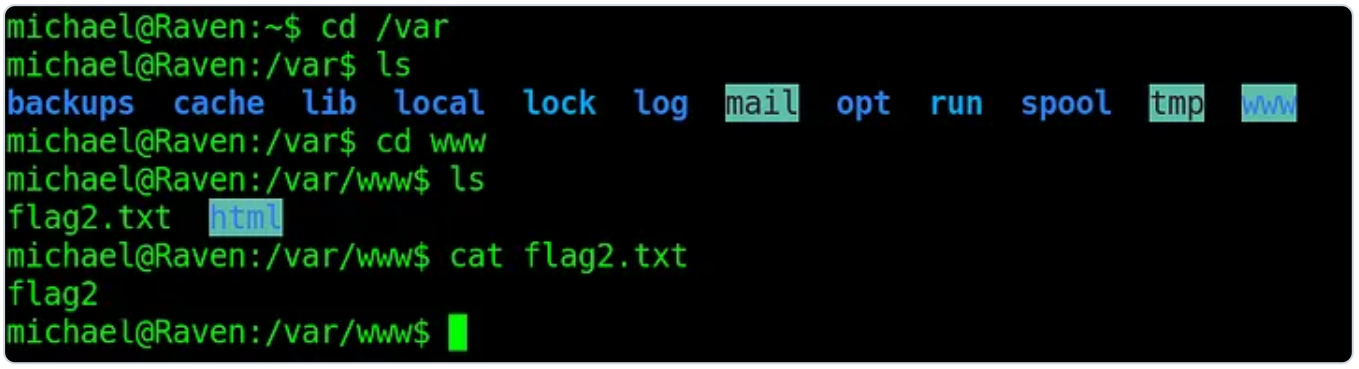
```
┌──(hari㉿hari)-[~]
└─$ ssh michael@192.168.29.167
michael@192.168.29.167's password:

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
You have mail.
Last login: Mon Feb  1 03:09:18 2021 from 192.168.29.223
michael@Raven:~$ mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 73
Server version: 5.5.60-0+deb8u1 (Debian)
```

Εικόνα 8 — Επιτυχής σύνδεση ως `michael` και είσοδος στη MySQL ως `root`.

Το επόμενο μεγάλο βήμα σε κάθε CTF, αφού αποκτήσουμε πρόσβαση ως απλοί χρήστες, είναι η **Κλιμάκωση Προνομίων** (Privilege Escalation) για να γίνουμε διαχειριστές (root). Όμως, προτού προχωρήσουμε σε αυτό, ας ρίξουμε μια ματιά στα διαθέσιμα αρχεία του συστήματος στα οποία έχουμε ήδη πρόσβαση.



Εικόνα 9 — Ανάγνωση του `flag2.txt` μέσα στο `/var/www/`.

Όταν ερευνούμε τους φακέλους, πλοηγούμαστε στον κατάλογο `/var/www/html/`. Μέσα σε αυτόν τον κατάλογο, εντοπίζουμε το **Δεύτερο Flag (Flag 2)**!

Βήμα 7: Εξερεύνηση Βάσης Δεδομένων (Εύρεση Flags 3 & 4)

Επεξήγηση Βήματος 7 — από αρχείο ρυθμίσεων σε βάση δεδομένων

Τι είναι η βάση δεδομένων. Το WordPress αποθηκεύει το περιεχόμενο και τους χρήστες του σε μια **σχεσιακή βάση δεδομένων** που διαχειρίζεται το σύστημα **MySQL**. Η εφαρμογή συνδέεται σε αυτήν χρησιμοποιώντας διαπιστευτήρια τα οποία είναι αποθηκευμένα στο αρχείο ρυθμίσεων `wp-config.php`.

Ο κρίσιμος ρόλος του `wp-config.php`. Το αρχείο αυτό αποτελεί κεντρικό σημείο ασφάλειας κάθε εγκατάστασης WordPress: περιέχει τις παραμέτρους `DB_NAME`, `DB_USER`, `DB_PASSWORD` και `DB_HOST`, καθώς και τα **κλειδιά αυθεντικοποίησης (authentication keys)**. Εάν ένας χρήστης αποκτήσει δικαίωμα ανάγνωσής του, διαθέτει πλέον τα διαπιστευτήρια για απευθείας πρόσβαση στη βάση δεδομένων.

Η αλυσίδα:

```
/wp-content/../wp-config.php  →  credentials  →  mysql -u user -p  →  USE wordpress;
                                                                    →  SELECT * FROM wp_posts;
```

Ανάλυση των SQL εντολών:

Εντολή	Τι κάνει
`SHOW DATABASES;`	Λίστα βάσεων — ψάχνουμε `wordpress`
`USE wordpress;`	Επιλογή βάσης (αλλιώς «No database selected»)
`SHOW TABLES;`	Πίνακες: `wp_posts`, `wp_users`, `wp_options`…
`SELECT * FROM wp_posts;`	Περιεχόμενο δημοσιεύσεων (εδώ κρύβονται flags)
`SELECT user_login, user_pass FROM wp_users;`	Χρήστες και **hashes**

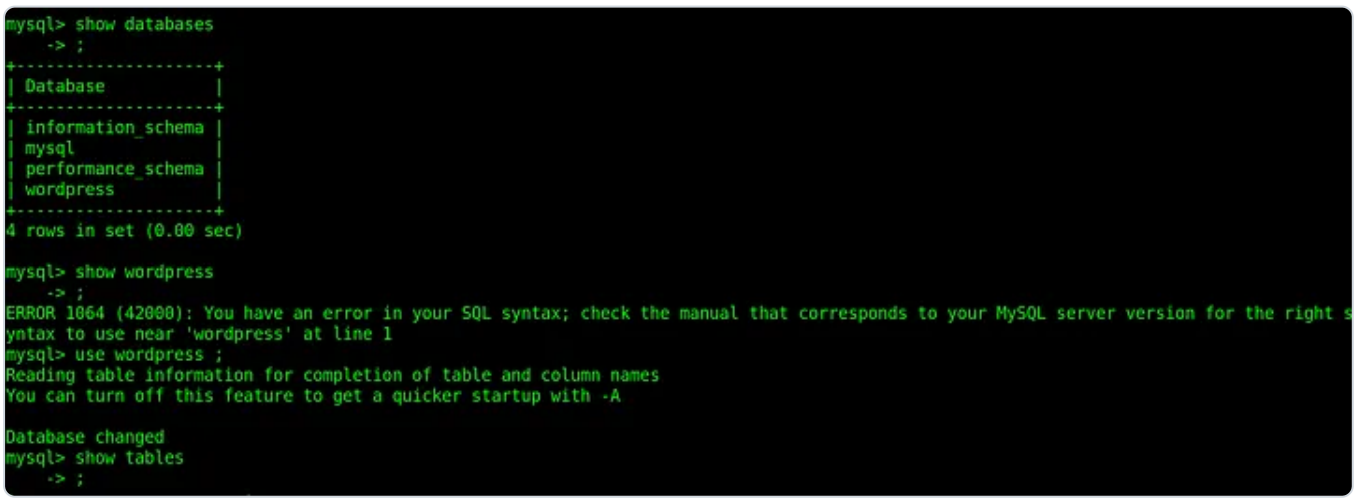
Γιατί τα flags βρίσκονται σε «drafts»: τα `wp_posts` έχουν πεδίο `post_status` (`publish`, `draft`, `inherit`). Τα flags εμφανίζονται σε εγγραφές **draft/revision** — δηλαδή σε περιεχόμενο που δεν δημοσιεύτηκε ποτέ, αλλά έμεινε στη βάση. Είναι ρεαλιστικό: διαγραμμένο ή κρυφό περιεχόμενο συχνά παραμένει.

Καλή πρακτική ελέγχου:

```
SELECT ID, post_title, post_status FROM wp_posts WHERE post_status ≠ 'publish';
SELECT user_login, user_pass FROM wp_users;
```

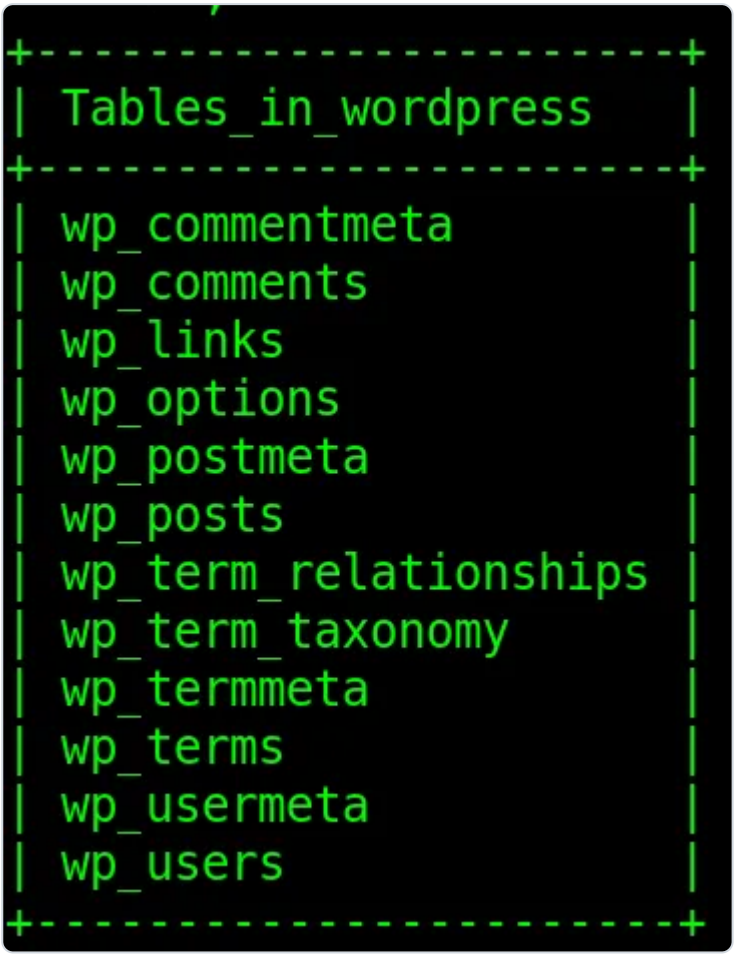
Γιατί αυτό είναι σοβαρό σε παραγωγή: ο web user (`www-data`) δεν θα έπρεπε να έχει δικαιώματα ανάγνωσης όλων των αρχείων του site, και η βάση δεν θα έπρεπε να εκθέτει hashes σε οποιονδήποτε έχει shell.
Κάνοντας μια ακόμη πιο βαθιά έρευνα στους καταλόγους, πηγαίνουμε στον φάκελο του WordPress (η διαδρομή είναι `/var/www/html/wordpress/`). Εκεί υπάρχει το αρχείο `wp-config.php`, το οποίο είναι εξαιρετικά σημαντικό καθώς περιέχει τα στοιχεία σύνδεσης της βάσης δεδομένων (Database credentials).

Χρησιμοποιούμε αυτά τα διαπιστευτήρια για να συνδεθούμε στη **MySQL** μέσω του τερματικού (π.χ. `mysql -u <χρήστης> -p`).



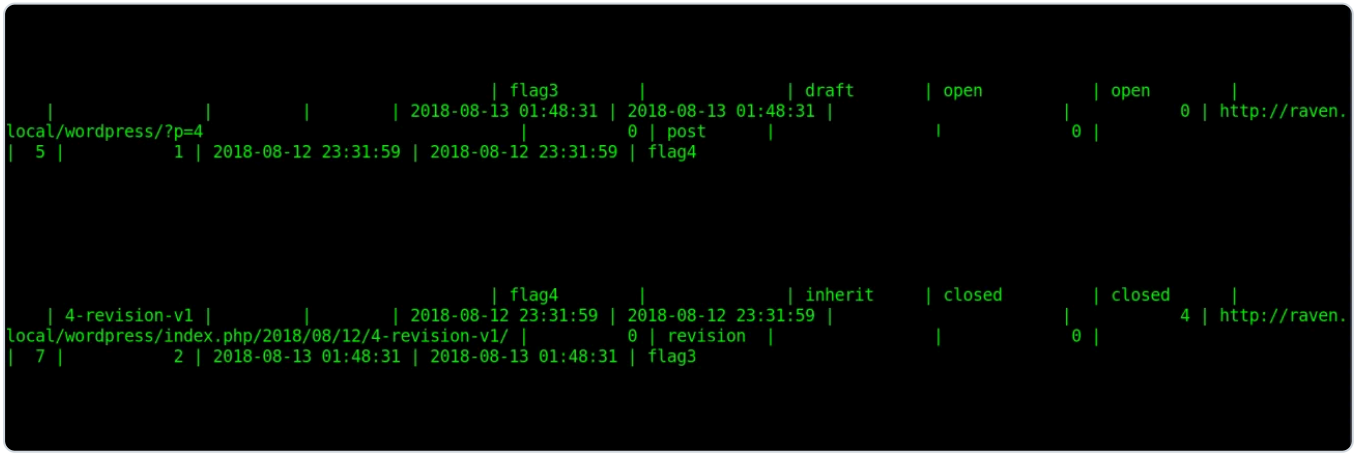
Εικόνα 10 — MySQL: βάσεις `mysql`, `performance_schema` και `wordpress`.

Αλλάζουμε την ενεργή βάση δεδομένων σε αυτή του WordPress (γράφοντας `USE wordpress;`). Τώρα, οι πίνακες (tables) της βάσης είναι ορατοί σε εμάς (`SHOW TABLES;`). Βλέπουμε κάποιους πολύ ενδιαφέροντες πίνακες, όπως ο πίνακας `wp_users` και ο `wp_posts`.



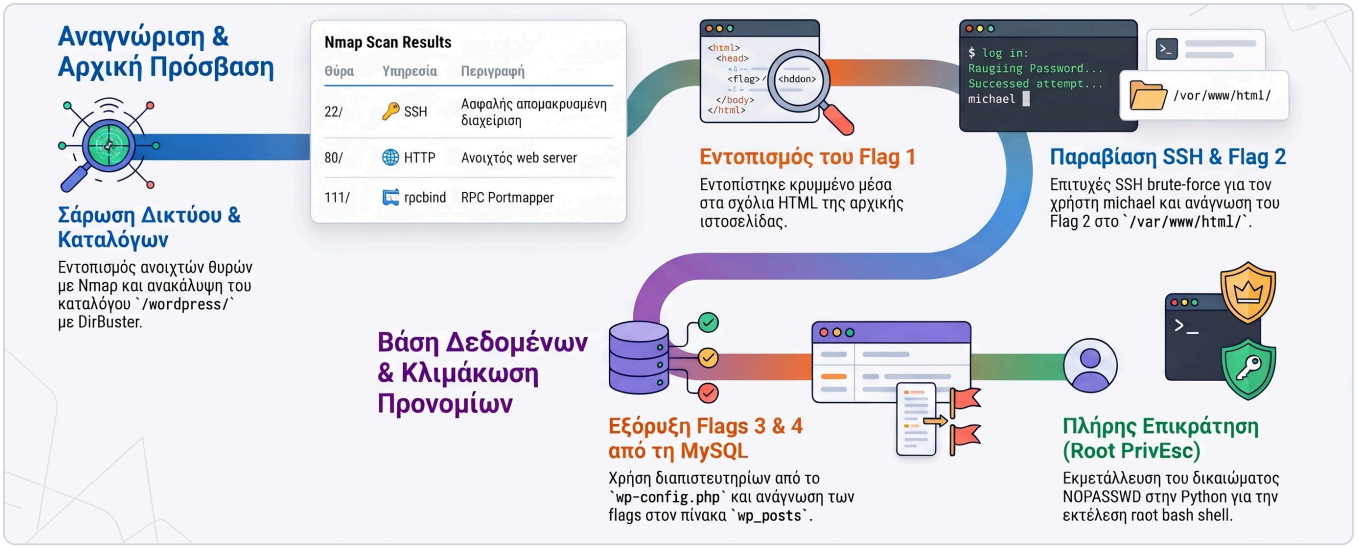
Εικόνα 11 — `SHOW TABLES`: οι πίνακες `wp_users`, `wp_posts`, `wp_options` κ.λπ.

Αν ψάξουμε μέσα στον πίνακα `wp_posts` (τις δημοσιεύσεις του site), θα βρούμε μέσα το **Τρίτο Flag (Flag 3)** και το **Τέταρτο Flag (Flag 4)**. Ουσιαστικά, αυτό αποτελεί και το επίσημο τέλος του CTF, αφού βρήκαμε και τις 4 σημαίες!



Εικόνα 12 — Στα `wp_posts` εμφανίζονται τα `flag3` και `flag4`.

Όμως μισό λεπτό... Τι γίνεται με το **Root Access** (την απόλυτη διαχειριστική πρόσβαση);



Βήμα 8: Ανάκτηση Κωδικού με John the Ripper

Επεξήγηση Βήματος 8 — η ανάκτηση ενός hash και οι κίνδυνοι της επαναχρησιμοποίησης κωδικών

Τι είναι μια συνάρτηση κατακερματισμού (hash). Πρόκειται για **μονόδρομη συνάρτηση** που μετατρέπει μια είσοδο οποιουδήποτε μήκους (τον κωδικό) σε μια συμβολοσειρά σταθερού μήκους (το hash). Η διαδικασία είναι μη αντιστρέψιμη: από το hash δεν είναι υπολογιστικά εφικτή η ανάκτηση του αρχικού κωδικού. Για τον λόγο αυτό τα συστήματα δεν αποθηκεύουν τους κωδικούς σε αναγνώσιμη μορφή, αλλά τα hashes τους. Ένα **salt** είναι μια τυχαία τιμή που προστίθεται στον κωδικό πριν τον κατακερματισμό, ώστε δύο ίδιοι κωδικοί να παράγουν διαφορετικά hashes και να αποτρέπεται η χρήση προϋπολογισμένων πινάκων (rainbow tables).

Η μορφή `$P$B …`. Το πρόθεμα `$P$` υποδηλώνει hash τύπου **phpass** — του αλγορίθμου που χρησιμοποιούσε το WordPress (βασίζεται σε MD5 με πολλαπλές επαναλήψεις και salt). Καθώς η συνάρτηση είναι μονόδρομη, το hash δεν «αποκρυπτογραφείται», ο κωδικός **ανακτάται** μέσω εξαντλητικής δοκιμής (διαδικασία γνωστή ως *cracking*).

Πώς λειτουργεί η επίθεση λεξικού. Το εργαλείο **John the Ripper** (`john`) εφαρμόζει την ακόλουθη λογική:

```
για κάθε λέξη στο rockyou.txt:
    υπολόγισε phpass(λέξη, salt)
    αν το αποτέλεσμα ταυτίζεται με το αποθηκευμένο hash  →  ο κωδικός βρέθηκε
```

Η διαδικασία είναι χρονοβόρα, διότι ο αλγόριθμος phpass έχει σχεδιαστεί ώστε να είναι **σκόπιμα βραδύς** (εκτελεί χιλιάδες επαναλήψεις ανά υπολογισμό, αυξάνοντας το κόστος κάθε δοκιμής). Το `john` αντισταθμίζει εν μέρει το κόστος αυτό εκτελώντας τους υπολογισμούς παράλληλα και με αλγοριθμικές βελτιστοποιήσεις.

Ανάλυση της εντολής:

```
john --wordlist=/usr/share/wordlists/rockyou.txt hashes.txt
john --show hashes.txt        # δείχνει ό,τι βρέθηκε
```

Γιατί ο κωδικός του `steven` ανοίγει και το root: επειδή ο διαχειριστής **επαναχρησιμοποίησε** τον κωδικό. Είναι το πιο συνηθισμένο λάθος σε πραγματικά περιστατικά: ένας κωδικός που διέρρευσε από εφαρμογή γίνεται κλειδί για το σύστημα.

Πώς θα έπρεπε να είναι: διαφορετικοί κωδικοί ανά υπηρεσία, `sudo` με NOPASSWD μόνο για συγκεκριμένες εντολές, MFA για SSH, και τακτικός έλεγχος με `john --test` / password policy.

Στην πράξη, μετά το `steven:pink84`: με τα διαπιστευτήρια αυτά συνδεόμαστε ως `steven` και αποκτούμε ένα κανονικό (μη προνομιούχο) κέλυφος, από το οποίο θα ξεκινήσει η κλιμάκωση προνομίων του επόμενου βήματος.

```
su steven          # ή: ssh steven@10.0.2.6   (κωδικός: pink84)
id                 # επιβεβαίωση: uid=1000(steven) – ακόμη όχι root
```

Σημείωση: το flag 4 δεν βρίσκεται πάντα στο `/root/flag4.txt` — σε αυτό το μηχάνημα η διαδρομή προς root είναι το ζητούμενο, και τα flags 1–3 έχουν ήδη βρεθεί μέσω web/DB.
Ο πίνακας `wp_users` που είδαμε προηγουμένως στη βάση δεδομένων, περιέχει τα "hashes" (κρυπτογραφημένες μορφές) των κωδικών πρόσβασης των χρηστών του WordPress.

```
mysql> select * from wp_users;
+----+------------+------------------------------------+---------------+-------------------
---+---------------------+
| ID | user_login | user_pass                          | user_nicename | user_email
us | display_name        |
+----+------------+------------------------------------+---------------+-------------------
---+---------------------+
|  1 | michael    | $P$BjRvZQ.VQcGZlDeiKToCQd.cPw5XCe0 | michael       | michael@raven.org
 0 | michael             |
|  2 | steven     | $P$Bk3VD9jsxx/loJoqNsURgHiaB23j7W/ | steven        | steven@raven.org
```

Εικόνα 13 — `SELECT * FROM wp_users`: τα hashes (phpass, `$P$…`) των `michael` και `steven`.

Αντιγράφουμε αυτά τα hashes και χρησιμοποιούμε ένα εργαλείο "σπασίματος" κωδικών που ονομάζεται **John The Ripper** (`john`) για να τα απορυπτογραφήσουμε (π.χ. παρέχοντας του ένα λεξικό όπως το `rockyou.txt`).
Η διαδικασία αυτή μας αποκαλύπτει με επιτυχία τον κωδικό πρόσβασης για τον χρήστη `steven`. Το πιο ενδιαφέρον κομμάτι είναι ότι ο κωδικός του `steven` τυγχάνει να είναι **ο ίδιος κωδικός που απαιτείται για την πρόσβαση στο root** (είτε μέσω `su root` είτε μέσω δικαιωμάτων `sudo`)!

```
# Αποθηκεύουμε τα hashes σε αρχείο και τρέχουμε λεξικό
john hashes.txt --wordlist=/usr/share/wordlists/rockyou.txt
```

```
Using default input encoding: UTF-8
Loaded 2 password hashes with 2 different salts (phpass [phpass ($ or $) 128/128 AVX 4x3])
Remaining 1 password hash
┌──(hari㉿hari)-[~]
└─$
```

Εικόνα 14 — John The Ripper: 2 hashes (phpass), απομένει 1 προς σπάσιμο.

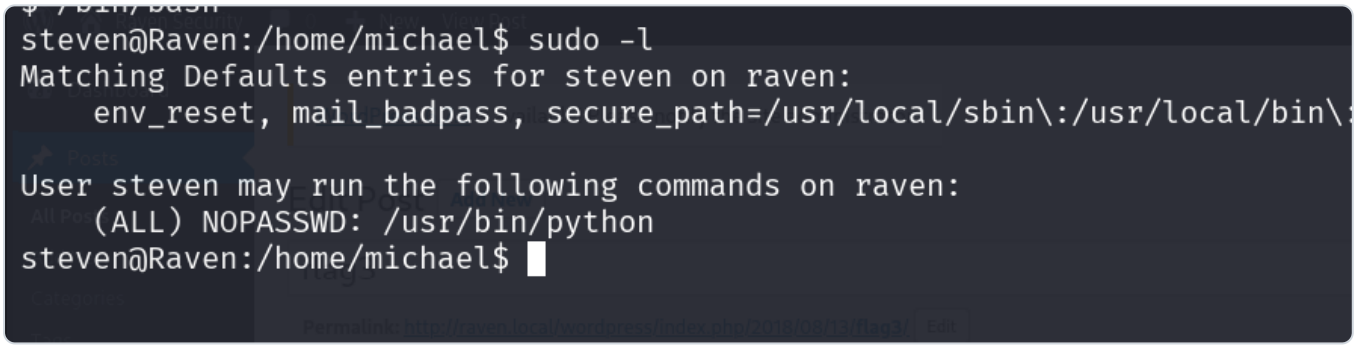


Εικόνα 15 — Επιτυχής ανάκτηση διαπιστευτηρίων: `steven:pink84`.

Πλέον έχουμε έγκυρα διαπιστευτήρια και πρόσβαση ως `steven`. Το τελευταίο βήμα είναι η **κλιμάκωση προνομίων** από τον χρήστη αυτόν προς `root`.

Βήμα 9: Ανύψωση Δικαιωμάτων με `sudo` + Python (GTFOBins)

Έχοντας πλέον κέλυφος ως ο χρήστης `steven`, το πρώτο πράγμα που εξετάζουμε για κλιμάκωση προνομίων είναι **τι μας επιτρέπεται να εκτελέσουμε ως `root`**. Αυτό το δείχνει η εντολή `sudo -l`, η οποία παραθέτει τις εγγραφές του αρχείου `/etc/sudoers` που αφορούν τον συνδεδεμένο χρήστη.



Έξοδος του τερματικού

```
Matching Defaults entries for steven on raven:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\ ...

User steven may run the following commands on raven:
    (ALL) NOPASSWD: /usr/bin/python
```

Παρατηρούμε ότι ο `steven` μπορεί να εκτελέσει την **Python ως `root` χωρίς κωδικό** (`NOPASSWD`). Αυτό από μόνο του είναι η ευπάθεια: η Python δεν είναι ένα «ασφαλές» εργαλείο που κάνει μία συγκεκριμένη δουλειά — είναι ένας **πλήρης διερμηνέας** που μπορεί να εκτελέσει αυθαίρετο κώδικα, άρα και να ξεκινήσει ένα κέλυφος. Εκμεταλλευόμαστε τη ρύθμιση καλώντας ένα διαδραστικό `bash` με δικαιώματα `root`:

```
sudo python -c 'import os; os.system("/bin/sh")'
```

```
sudo python -c 'import os; os.system("/bin/bash")'
```

Η τεχνική αυτή (καταγεγραμμένη στο **GTFOBins**) αξιοποιεί το δικαίωμα εκτέλεσης της Python ως `root` ώστε να «δραπετεύσει» από το περιβάλλον του διερμηνέα (**binary escape**) και να ανοίξει ένα πλήρες root shell (`root@raven`).

Επεξήγηση Βήματος 9 — γιατί ένα απλό `sudo python` δίνει root

Τι είναι το GTFOBins. Πρόκειται για μια δημόσια βάση γνώσης (*Get The F* Out Binaries*) που καταγράφει πώς «νόμιμα» εκτελέσιμα του Unix μπορούν να καταχραστούν όταν τους δοθούν αυξημένα δικαιώματα — μέσω `sudo` , `SUID` , δυνατοτήτων (capabilities) κ.ο.κ. Η λογική πίσω από κάθε εγγραφή είναι η ίδια: αν ένα πρόγραμμα που τρέχει ως `root` μπορεί να εκτελέσει **εντολές ή κώδικα της επιλογής μας**, τότε τα προνόμιά του γίνονται δικά μας.

Ανάλυση της εντολής:

- `sudo python` — εκκινεί τον διερμηνέα της Python υπό τον λογαριασμό του υπερχρήστη. Λόγω του `NOPASSWD` , δεν ζητείται καν κωδικός.
- `-c` (*command*) — επιτρέπει τη μετάδοση κώδικα Python απευθείας ως όρισμα, χωρίς να χρειάζεται ξεχωριστό αρχείο `.py` .
- `import os` — εισάγει το ενσωματωμένο module `os` , που παρέχει διεπαφές προς το λειτουργικό σύστημα.
- `os.system("/bin/bash")` — καλεί τη συνάρτηση συστήματος, εκτελώντας το κέλυφος `/bin/bash` .

Ο μηχανισμός της κλιμάκωσης. Όταν τρέχουμε κάτι με `sudo` , η διεργασία αποκτά **effective UID 0** (root). Η Python είναι πλέον μια διεργασία-γονέας με δικαιώματα root. Η `os.system()` κάνει `fork()` και εκκινεί το `/bin/bash` ως **διεργασία-παιδί**, κάθε παιδί **κληρονομεί** το περιβάλλον ασφαλείας του γονέα, άρα το νέο κέλυφος ξεκινά κι αυτό ως `root` . Δεν υπάρχει εδώ «σπάσιμο» κάποιου μηχανισμού ασφαλείας: απλώς μια υπερβολικά πλατιά άδεια (`sudo` σε έναν ολόκληρο διερμηνέα) χρησιμοποιείται ακριβώς για ό,τι επιτρέπει τεχνικά.

Επαλήθευση. Μέσα στο νέο κέλυφος εκτελούμε:

```
id        # uid=0(root) gid=0(root) groups=0(root)
whoami    # root
```

Αν το `id` επιστρέψει `uid=0(root)` , η κλιμάκωση ολοκληρώθηκε και έχουμε πλήρη έλεγχο του μηχανήματος (και πρόσβαση στο `/root/flag4.txt` , όπου υπάρχει).

Σημείωση — εναλλακτική διατύπωση. Η ίδια ιδέα γράφεται και ως `sudo python -c 'import pty; pty.spawn("/bin/bash")'` . Το module `pty` δίνει ένα πλήρως **διαδραστικό** ψευδο-τερματικό (χρήσιμο για job control, `sudo` , interactive προγράμματα), ενώ η `os.system` αρκεί για ένα απλό root shell. Και οι δύο βασίζονται στο ίδιο κενό ρύθμισης.

Πώς θα έπρεπε να είναι (αντιμετώπιση). Η ρίζα του προβλήματος είναι η παραβίαση της **αρχής των ελάχιστων προνομίων**: ποτέ δεν δίνουμε `sudo` σε έναν γενικό διερμηνέα (Python, Perl, Ruby) ή σε εργαλεία που μπορούν να εκκινήσουν κέλυφος (`vi` , `less` , `find` , `awk` , `tar`). Αν χρειάζεται

αυτοματισμός, δίνουμε δικαίωμα μόνο στο **συγκεκριμένο script** με πλήρη διαδρομή και ελεγμένες παραμέτρους, κατά προτίμηση χωρίς `NOPASSWD`.

Σύνοψη

Επεξήγηση — τι να κρατήσετε από αυτό το εργαστήριο

Φάση	Εργαλείο	Κρίσιμη ιδέα
Recon	`netdiscover`	Το ARP δεν «φιλτράρεται» σε LAN
Scan	`nmap -sC -sV`	Οι εκδόσεις υπηρεσιών οδηγούν σε γνωστές ευπάθειες
Web	DevTools / `curl`	Τα σχόλια HTML διαρρέουν πληροφορία
Fuzzing	DirBuster / `gobuster`	Τα 403 είναι ευρήματα, όχι αποτυχίες
CMS	`wpscan -e u`	Οι χρήστες είναι το ήμισυ του προβλήματος
Access	`hydra` + λεξικό	Οι «προφανείς» κωδικοί δουλεύουν
Post	`wp-config.php`	Τα credentials γράφονται σε αρχεία
Privesc	`sudo -l` + Python (GTFOBins)	`sudo` σε διερμηνέα = root, η `john` + επαναχρήση κωδικού είναι εναλλακτική
Η μεθοδολογία που ακολουθήθηκε είναι, ως προς τη σύλληψή της, σχετικά απλή, η κύρια δυσκολία για τον αρχάριο αναλυτή έγκειται στον προσδιορισμό του ορθού πρώτου βήματος και στη μετάβαση από τη μία φάση στην επόμενη. Οι δυσκολίες αυτές είναι αναμενόμενες κατά την επίλυση ενός CTF, με συστηματική εξάσκηση, η εξοικείωση με τις τεχνικές και τα εργαλεία βελτιώνεται σταθερά.		

Το μηχάνημα διαθέτει και συνέχεια (**Raven 2**), η οποία καλύπτεται σε αντίστοιχο εργαστήριο του παρόντος υλικού (Εργαστήριο 04).

Παράρτημα Α — Χάρτης εντολών του εργαστηρίου

Σκοπός	Εντολή
Εύρεση IP στόχου	`netdiscover -r 10.0.2.0/24`
Σάρωση υπηρεσιών	`nmap -sC -sV 10.0.2.6`
Έλεγχος headers	`curl -I http://10.0.2.6/`
Εύρεση καταλόγων	`dirb http://10.0.2.6/` ή `gobuster dir -u … -w …`
Απαρίθμηση WP	`wpscan --url http://10.0.2.6/wordpress -e u`
SSH brute-force	`hydra -l michael -P rocklyn.txt ssh://10.0.2.6`
Σύνδεση	`ssh michael@10.0.2.6`
Δικαιώματα	`id`, `sudo -l`, `ls -la /var/www`
MySQL	`mysql -u root -p` → `USE wordpress; SHOW TABLES;`
Σπάσιμο hashes	`john --wordlist=rockyou.txt hashes.txt`

Παράρτημα Β — Γλωσσάρι

Όρος	Σημασία
Reconnaissance	Συλλογή πληροφοριών χωρίς άμεση επαφή/επίθεση
Enumeration	Ενεργητική απαρίθμηση (χρήστες, κατάλογοι, εκδόσεις)
Fuzzing (dir)	Δοκιμή λίστας ονομάτων για ανακάλυψη πόρων
CMS	Σύστημα διαχείρισης περιεχομένου (WordPress, Drupal)
Hash	Μονόδρομη συνάρτηση, δεν αντιστρέφεται — ο κωδικός ανακτάται μόνο μέσω εξαντλητικών δοκιμών
phpass	Αλγόριθμος hashing του WordPress (`$P$`)
Salt	Τυχαία τιμή που κάνει κάθε hash μοναδικό
Credential reuse	Ίδιος κωδικός σε πολλά συστήματα
Privilege escalation	Από απλός χρήστης σε root

Παράρτημα Γ — Ερωτήσεις αυτοελέγχου

1. Γιατί το `netdiscover` βρίσκει hosts που το `ping` δεν βλέπει;
2. Ποια η διαφορά `open` / `closed` / `filtered` σε ένα nmap;

3. Γιατί η απαρίθμηση χρηστών στο WordPress διευκολύνει το brute-force;
4. Τι σημαίνει HTTP 403 για έναν κατάλογο και γιατί είναι χρήσιμο;
5. Γιατί δεν «αποκρυπτογραφείται» ένα hash; Περιγράψτε τη διαδικασία του cracking.
6. Ποια τρία μέτρα θα σταματούσαν την πορεία αυτού του CTF;
7. Πού θα ψάξετε πρώτα μετά την απόκτηση shell και γιατί (`sudo -l` , `id` , `ls /var/www`);

Ενδεικτικές απαντήσεις

1. Το `netdiscover` δουλεύει στο **επίπεδο 2 (ARP)**: κάθε ζωντανός host **πρέπει** να απαντά σε ARP για να λειτουργεί το δίκτυο. Το `ping` χρησιμοποιεί **ICMP (επίπεδο 3)**, το οποίο πολλά συστήματα/firewalls μπλοκάρουν — έτσι ένας host που αγνοεί τα ICMP echo φαίνεται «νεκρός» στο ping αλλά εντοπίζεται από το ARP. (Ισχύει μόνο στο ίδιο LAN, όπου το ARP δεν περνά routers.)
2. **open** = υπάρχει υπηρεσία που ακούει και απάντησε (SYN-ACK). **closed** = η θύρα είναι προσβάσιμη αλλά καμία υπηρεσία δεν ακούει (απάντησε με RST). **filtered** = δεν ήρθε απάντηση ή ήρθε ICMP unreachable — κάτι (συνήθως firewall) φιλτράρει το πακέτο, οπότε δεν ξέρουμε αν είναι open ή closed.
3. Το brute-force χρειάζεται **έγκυρο username + κωδικό**. Αν ξέρουμε τους πραγματικούς χρήστες (π.χ. `michael` , `steven` μέσω `wpscan -e u`), ο επιτιθέμενος σταματά να μαντεύει usernames και ρίχνει όλες τις δοκιμές στον κωδικό ενός γνωστού λογαριασμού — μειώνει δραστικά τον χώρο αναζήτησης.
4. 403 Forbidden = ο κατάλογος **υπάρχει** αλλά απαγορεύεται η πρόσβαση/listing. Είναι θετικό εύρημα: επιβεβαιώνει έγκυρη διαδρομή (σε αντίθεση με 404), άρα αξίζει να δοκιμάσουμε γνωστά ονόματα αρχείων μέσα της. Τα 403 στο fuzzing είναι «στόχοι», όχι αποτυχίες.
5. Το hash παράγεται από **μονόδρομη** συνάρτηση — δεν υπάρχει αντίστροφη πράξη. Το cracking δεν αποκρυπτογραφεί, **μαντεύει**: παίρνει υποψήφιους κωδικούς (rockyou), υπολογίζει το ίδιο hash με το ίδιο salt/αλγόριθμο (phpass) και συγκρίνει με το αποθηκευμένο, όταν ταιριάξουν, βρέθηκε ο κωδικός. Γι' αυτό αργοί αλγόριθμοι + salt κάνουν το cracking ακριβό.
6. (1) **Ισχυροί κωδικοί + lockout**: ο `michael:michael` δεν θα έσπαγε με hydra με password policy και κλείδωμα μετά από X αποτυχίες. (2) **Προστασία credentials**: `wp-config.php` με αυστηρά δικαιώματα και MySQL δεμένη στο `127.0.0.1` κρύβουν τα διαπιστευτήρια της βάσης. (3) **Least privilege στο sudo**: αφαίρεση του `NOPASSWD: /usr/bin/python` κόβει την τελική κλιμάκωση σε root.
7. Πρώτα `id` / `whoami` (ποιος είμαι, σε ποια groups), μετά `sudo -l` (τι τρέχω ως root — συχνά η ταχύτερη διαδρομή privesc) και `ls -la /var/www` + ανάγνωση config (`wp-config.php`) για credentials/flags. Η λογική: χαρτογραφούμε άμεσα προνόμια και «εύκολα» μυστικά πριν ψάξουμε πιο σύνθετα μονοπάτια (SUID, cron, kernel).

Παράρτημα Δ — Λίστα ελέγχου

- [ ] Επιβεβαίωση IP στόχου (ARP, όχι μόνο ping)
- [ ] Σάρωση **όλων** των θυρών (τουλάχιστον `-sV` και, αν χρειάζεται, `-p-`)
- [ ] Έλεγχος πηγαίου κώδικα, headers, `robots.txt`
- [ ] Directory fuzzing με λεξικό

- [ ] Απαρίθμηση χρηστών CMS
- [ ] Έλεγχος αδύναμων/επαναχρησιμοποιημένων κωδικών
- [ ] Ανάγνωση config αρχείων για credentials
- [ ] Καταγραφή flags και διαδρομής επίλυσης (για το report)

Raven 1 — Εγχειρίδιο Θωράκισης (Hardening & Remediation Guide)

Το README.md έδειξε **πώς** παραβιάστηκε το Raven 1. Αυτό το εγχειρίδιο εξηγεί **γιατί** ήταν δυνατή κάθε παραβίαση και **πώς** τη διορθώνουμε, μόνο με ρυθμίσεις, αναβαθμίσεις και αλλαγές δικαιωμάτων στο ίδιο το σύστημα.

Η λογική του εγχειριδίου είναι η εξής: κάθε βήμα της επίθεσης άνοιξε επειδή υπήρχε **ένα λάθος στις ρυθμίσεις**. Αν διορθώσουμε το λάθος, το αντίστοιχο βήμα δεν γίνεται. Αν διορθώσουμε **περισσότερα από ένα** λάθη στην ίδια αλυσίδα, η επίθεση σταματά ακόμη κι αν μία από τις διορθώσεις αποτύχει. Αυτό λέγεται **Άμυνα σε Βάθος (Defense in Depth)**.

> ⚠️ **Πριν ξεκινήσετε:** πάρτε **snapshot** της VM (βλ. εργαστήριο 00). Έτσι μπορείτε να επιστρέψετε στην ευάλωτη κατάσταση, να ξανατρέξετε την επίθεση και να δείτε ότι πλέον αποτυγχάνει. Αυτή η σύγκριση «πριν/μετά» είναι το πιο χρήσιμο κομμάτι της άσκησης.
>
> ℹ️ Όλες οι εντολές εκτελούνται **στη μηχανή Raven** ως `root` (π.χ. μετά από `su -`). Οι IP διευθύνσεις είναι ενδεικτικές.

Περιεχόμενα

1. Χάρτης: Επίθεση → Αιτία → Διόρθωση
2. Απογραφή του συστήματος (τι έχουμε στα χέρια μας)
3. Αναβάθμιση λειτουργικού και πακέτων
4. Ελαχιστοποίηση επιφάνειας επίθεσης: υπηρεσίες & firewall
5. Θωράκιση Apache (σχόλια HTML, listing, banners)
6. Θωράκιση WordPress (έκδοση, απαρίθμηση χρηστών, wp-config)
7. Θωράκιση SSH & πολιτική κωδικών (hydra, michael:michael)
8. Θωράκιση MySQL (root χωρίς περιορισμούς)
9. Διόρθωση sudo (NOPASSWD python)
10. Hashes κωδικών & επαναχρησιμοποίηση κωδικών (John the Ripper)
11. Επαλήθευση: ξανατρέχουμε την επίθεση
12. Λίστα ελέγχου & ερωτήσεις αυτοελέγχου

---

1. Χάρτης: Επίθεση → Αιτία → Διόρθωση

Βήμα επίθεσης (README)	Τι εκμεταλλεύτηκε ο επιτιθέμενος	Ρίζα του προβλήματος	Διόρθωση (ενότητα)
2. `nmap`	Θύρες 22, 80, 111 ανοιχτές προς όλους	Περιττές υπηρεσίες, κανένα firewall	§4
3. Flag 1 στο HTML	Σχόλιο `<!— flag —>` στον κώδικα σελίδας	Ευαίσθητες πληροφορίες στον client	§5
4. DirBuster	Ανακάλυψη `/wordpress/`, `/vendor/`	Directory listing, αρχεία ανάπτυξης στο web root	§5
5. WPScan	Έκδοση WP, χρήστες `michael`, `steven`	Παλιό WordPress, ανοιχτή απαρίθμηση χρηστών	§3, §6
6. `hydra` → SSH	`michael:michael`	Κανένα όριο προσπαθειών, password login, αδύναμος κωδικός	§7
7. `wp-config.php` → MySQL `root`	Κωδικός root της βάσης σε αναγνώσιμο αρχείο	Η εφαρμογή χρησιμοποιεί τον `root` της MySQL	§6, §8
8. `sudo python`	`(ALL) NOPASSWD: /usr/bin/python`	Λάθος κανόνας sudoers	§9
8b. `john`	Hash του `steven` σπάει με `rockyou.txt`	Αδύναμος κωδικός, επαναχρησιμοποίηση κωδικών	§10

> 💡 Παρατηρήστε ότι **κανένα** βήμα δεν χρειάστηκε κάποιο «προχωρημένο» exploit. Όλα ήταν **λάθη ρυθμίσεων** (misconfigurations). Γι' αυτό η διόρθωσή τους είναι κυρίως θέμα πειθαρχίας και όχι κόστους.

---

2. Απογραφή του συστήματος

Δεν μπορούμε να προστατέψουμε κάτι που δεν ξέρουμε ότι υπάρχει. Ο διαχειριστής ξεκινά πάντα με την ίδια ερώτηση που έκανε ο επιτιθέμενος: **τι τρέχει σε αυτό το μηχάνημα;**

```
# Ποια διανομή και έκδοση έχουμε;
cat /etc/os-release
uname -r

# Ποιες υπηρεσίες ακούνε σε θύρες δικτύου;
ss -tulpn
```

- `cat /etc/os-release`: δείχνει τη διανομή. Στο Raven 1 είναι **Debian 8 (Jessie)**, έκδοση που δεν λαμβάνει πλέον ενημερώσεις ασφαλείας.
- `uname -r`: η έκδοση του πυρήνα (kernel). Παλιός πυρήνας σημαίνει γνωστές ευπάθειες τοπικής κλιμάκωσης προνομίων.
- `ss -tulpn`: το ίδιο που βλέπει το `nmap`, αλλά **από μέσα**. Οι σημαίες σημαίνουν: `-t` TCP, `-u` UDP, `-l` μόνο θύρες σε αναμονή (listening), `-p` ποια διεργασία την κατέχει, `-n` αριθμοί αντί για ονόματα.

Κοιτάξτε τη στήλη `Local Address`:

Τιμή	Σημασία
`0.0.0.0:22` ή `*:22`	Ακούει σε **όλες** τις διεπαφές, άρα είναι προσβάσιμο από το δίκτυο
`127.0.0.1:3306`	Ακούει **μόνο τοπικά**, δεν φαίνεται από έξω (αυτό θέλουμε για τη MySQL)

```
# Ποιοι χρήστες έχουν shell σύνδεσης;
grep -vE 'nologin|false' /etc/passwd

# Ποιοι κανόνες sudo υπάρχουν;
cat /etc/sudoers; ls -la /etc/sudoers.d/
```

Καταγράψτε τα αποτελέσματα. Θα τα συγκρίνετε με την κατάσταση μετά τη θωράκιση.

---

3. Αναβάθμιση λειτουργικού και πακέτων

3.1 Γιατί έχει σημασία

Το WPScan αναγνώρισε την ακριβή έκδοση του WordPress και τη συσχέτισε με γνωστές ευπάθειες (CVE). Το ίδιο ισχύει για Apache, PHP, MySQL και τον πυρήνα. **Μια γνωστή ευπάθεια σε παλιό λογισμικό είναι ο πιο φθηνός τρόπος παραβίασης**, γιατί το exploit υπάρχει ήδη δημόσια.

Ο κύκλος ζωής μιας διανομής έχει σημασία:

Διανομή	Κατάσταση
Debian 8 Jessie (Raven 1)	**Τέλος υποστήριξης**. Τα αποθετήρια έχουν μεταφερθεί στο `archive.debian.org`
Debian 12 Bookworm / 13 Trixie	Υποστηρίζονται, λαμβάνουν security updates

3.2 Άμεση ενημέρωση εντός της ίδιας έκδοσης

Επειδή το Jessie είναι αρχειοθετημένο, το `apt update` αποτυγχάνει με τα αρχικά αποθετήρια. Τα στρέφουμε στο αρχείο:

```
cp /etc/apt/sources.list /etc/apt/sources.list.bak
cat > /etc/apt/sources.list <<'EOF'
deb http://archive.debian.org/debian jessie main
deb http://archive.debian.org/debian-security jessie/updates main
EOF

apt-get -o Acquire::Check-Valid-Until=false update
apt-get upgrade
```

- Κρατάμε αντίγραφο (`.bak`) πριν αλλάξουμε αρχείο ρυθμίσεων. Είναι καλή συνήθεια για κάθε αλλαγή.
- `Acquire::Check-Valid-Until=false` : τα αρχειοθετημένα αποθετήρια έχουν «ληγμένη» υπογραφή χρόνου. Η παράμετρος λέει στο `apt` να τη δεχτεί. **Αυτό είναι αποδεκτό μόνο σε εργαστήριο**.
- `apt-get upgrade` : εγκαθιστά τις τελευταίες εκδόσεις που υπήρξαν ποτέ για το Jessie.

3.3 Η πραγματική λύση: μετάβαση σε υποστηριζόμενη έκδοση

Η ενημέρωση εντός του Jessie καλύπτει **ορισμένα** μόνο κενά ασφαλείας, όχι το σύνολό τους, διότι για τις ευπάθειες που εντοπίζονται μετά το τέλος υποστήριξης δεν πρόκειται να εκδοθεί ποτέ διόρθωση. Σε παραγωγικό περιβάλλον, η ορθή απόφαση είναι η **νέα εγκατάσταση** σε Debian 12/13 και η μεταφορά της εφαρμογής (αρχεία και αντίγραφο της βάσης — database dump). Η σταδιακή, διαδοχική αναβάθμιση (Jessie → Stretch → Buster → Bullseye → Bookworm) είναι εφικτή, αλλά επίπονη και επιρρεπής σε σφάλματα.

3.4 Αυτόματες ενημερώσεις ασφαλείας (σε υποστηριζόμενο σύστημα)

```
apt install unattended-upgrades
dpkg-reconfigure -plow unattended-upgrades
```

Το πακέτο εγκαθιστά καθημερινά **μόνο** τις ενημερώσεις ασφαλείας. Έτσι το κενό μεταξύ «δημοσίευσης διόρθωσης» και «εφαρμογής της» μειώνεται από μήνες σε ώρες.

---

4. Ελαχιστοποίηση επιφάνειας επίθεσης

4.1 Απενεργοποίηση του rpcbind (θύρα 111)

Το `rpcbind` χρειάζεται μόνο για υπηρεσίες RPC όπως το **NFS**. Ο web server δεν τις χρησιμοποιεί. Κάθε υπηρεσία που δεν χρειάζεται είναι επιπλέον κώδικας που μπορεί να έχει ευπάθεια και επιπλέον πληροφορία για τον επιτιθέμενο.

```
# Βεβαιωνόμαστε ότι δεν υπάρχουν NFS mounts που το χρειάζονται
mount | grep nfs

# Απενεργοποίηση (Debian 8 χρησιμοποιεί systemd)
systemctl stop rpcbind.socket rpcbind.service
systemctl disable rpcbind.socket rpcbind.service
systemctl mask rpcbind.service
```

- `stop`: σταματά την υπηρεσία τώρα.
- `disable`: δεν ξεκινά στην επόμενη εκκίνηση.
- `mask`: την «κλειδώνει», ώστε να μην μπορεί να ξεκινήσει ούτε ως εξάρτηση άλλης υπηρεσίας.
- Απενεργοποιούμε και το `.socket`, γιατί με το socket activation το systemd θα ξεκινούσε ξανά την υπηρεσία μόλις έρθει σύνδεση στη θύρα 111.

Αν η υπηρεσία δεν χρειάζεται καθόλου: `apt purge rpcbind nfs-common`.

4.2 Firewall με πολιτική «απαγόρευση εκτός αν επιτρέπεται»

Η σωστή φιλοσοφία είναι το **default deny**: κλείνουμε τα πάντα και ανοίγουμε μόνο ό,τι χρειάζεται.

```
apt install ufw

ufw default deny incoming       # Όλα τα εισερχόμενα απορρίπτονται
ufw default allow outgoing      # Τα εξερχόμενα επιτρέπονται (θα το σφίξουμε αργότερα αν χρειαστεί)

ufw allow 80/tcp                # HTTP για το site
ufw allow 443/tcp               # HTTPS (μόλις ρυθμιστεί πιστοποιητικό)

# SSH μόνο από το υποδίκτυο διαχείρισης, ΟΧΙ από παντού
ufw allow from 192.168.56.0/24 to any port 22 proto tcp

ufw enable
ufw status verbose
```

> ⚠️ **Προσοχή στη σειρά των ενεργειών.** Εάν η διαχείριση γίνεται μέσω SSH, ο κανόνας για τη θύρα 22 πρέπει να προστεθεί **πριν** από την εντολή `ufw enable`, σε αντίθετη περίπτωση θα χαθεί η πρόσβαση στο μηχάνημα (αποκλεισμός — lockout).

Γιατί περιορίζουμε το SSH σε υποδίκτυο; Ο επιτιθέμενος έτρεξε το `hydra` από το ίδιο δίκτυο με το site. Αν το SSH δεχόταν συνδέσεις μόνο από ένα δίκτυο διαχείρισης, η επίθεση brute-force δεν θα ξεκινούσε καν.

Επιπλέον, το `ufw` έχει ενσωματωμένο **περιορισμό ρυθμού** (rate limiting):

```
ufw limit 22/tcp
```

Το `limit` απορρίπτει μια IP που ανοίγει **6 ή περισσότερες συνδέσεις μέσα σε 30 δευτερόλεπτα**. Το `hydra` ανοίγει δεκάδες παράλληλες συνδέσεις, άρα θα κοβόταν αμέσως.

---

5. Θωράκιση Apache

5.1 Ευαίσθητα σχόλια στον κώδικα HTML (Flag 1)

Το πρόβλημα: ό,τι στέλνει ο server στον browser το βλέπει **ο καθένας** με `F12` ή `Ctrl+U`. Τα σχόλια HTML (`<!-- -->`) δεν είναι κρυφά, απλώς δεν εμφανίζονται στη σελίδα.

Έλεγχος: ψάχνουμε σχόλια σε όλα τα στατικά αρχεία:

```
grep -rn '<!--' /var/www/html --include=*.html --include=*.php | less
```

Διόρθωση: αφαιρούμε κάθε σχόλιο που περιέχει πληροφορίες (κωδικούς, διαδρομές, ονόματα χρηστών, σημειώσεις προγραμματιστών). Σε μια σωστή διαδικασία ανάπτυξης, τα σχόλια αφαιρούνται αυτόματα στο στάδιο «build» πριν το ανέβασμα στον server.

5.2 Directory Listing (DirBuster)

Το πρόβλημα: όταν ένας κατάλογος δεν έχει `index.html` / `index.php`, ο Apache με την επιλογή `Indexes` εμφανίζει **λίστα όλων των αρχείων** του. Ο επιτιθέμενος βλέπει όλη τη δομή χωρίς καν να μαντέψει ονόματα.

Έλεγχος:

```
grep -rn "Indexes" /etc/apache2/
curl -s http://localhost/vendor/ | head
```

Αν το `curl` επιστρέψει `Index of /vendor`, το listing είναι ενεργό.

Διόρθωση: στο `/etc/apache2/apache2.conf`, στο μπλοκ του `/var/www/`:

```
<Directory /var/www/>
    Options -Indexes +FollowSymLinks
    AllowOverride All
    Require all granted
</Directory>
```

- `-Indexes`: το πρόσημο `-` **αφαιρεί** την επιλογή. Ο Apache πλέον απαντά `403 Forbidden` αντί για λίστα αρχείων.
- `AllowOverride All`: χρειάζεται ώστε το WordPress να χρησιμοποιεί `.htaccess` για τα permalinks.

5.3 Αρχεία ανάπτυξης μέσα στο web root (`/vendor/`)

Ο κατάλογος `vendor/` (βιβλιοθήκες Composer, π.χ. PHPMailer) **δεν πρέπει ποτέ** να είναι προσβάσιμος από τον browser. Το ίδιο ισχύει για `.git/`, `composer.json`, `*.bak`, `*.sql`.

```
# /etc/apache2/conf-available/deny-dev-files.conf
<DirectoryMatch "/(vendor|\.git|node_modules)/">
    Require all denied
</DirectoryMatch>

<FilesMatch "\.(bak|sql|old|swp|log|md|json|lock)$">
    Require all denied
</FilesMatch>
```

```
a2enconf deny-dev-files
```

Ακόμα καλύτερη λύση είναι να **μετακινηθεί** το `vendor/` έξω από το `/var/www/html` και να φορτώνεται από τον κώδικα PHP με απόλυτη διαδρομή.

5.4 Απόκρυψη εκδόσεων (banners)

Το `nmap -sV` και οι σελίδες σφάλματος του Apache αποκαλύπτουν εκδόσεις (`Apache/2.4.10 (Debian)`). Η απόκρυψη **δεν διορθώνει** καμία ευπάθεια, αλλά κάνει την αναγνώριση πιο αργή και θορυβώδη.

```
# /etc/apache2/conf-available/security.conf
ServerTokens Prod        # Header: "Server: Apache" χωρίς έκδοση/OS
ServerSignature Off      # Χωρίς υπογραφή στο κάτω μέρος των σελίδων σφάλματος
TraceEnable Off          # Απενεργοποιεί τη μέθοδο HTTP TRACE

Header always set X-Content-Type-Options "nosniff"
Header always set X-Frame-Options "SAMEORIGIN"
```

```
# Το ίδιο και για την PHP: κρύβει το header "X-Powered-By: PHP/5.x"
sed -i 's/^expose_php = On/expose_php = Off/' /etc/php5/apache2/php.ini

a2enmod headers
apache2ctl configtest      # ΠΑΝΤΑ έλεγχος σύνταξης πριν την επανεκκίνηση
systemctl reload apache2
```

> 💡 Η εντολή `apache2ctl configtest` πρέπει να επιστρέφει `Syntax OK`. Εάν εκτελεστεί `reload` με εσφαλμένη ρύθμιση, ο Apache ενδέχεται να μην εκκινήσει και ο ιστότοπος να τεθεί εκτός λειτουργίας.

---

6. Θωράκιση WordPress

6.1 Αναβάθμιση πυρήνα, plugins, themes

Το WordPress του Raven 1 είναι έκδοση **4.8.x** (2017). Το WPScan έδειξε δεκάδες γνωστές ευπάθειες για αυτή την έκδοση.

Με το **WP-CLI** (το επίσημο εργαλείο γραμμής εντολών):

```
# Εγκατάσταση WP-CLI
curl -O https://raw.githubusercontent.com/wp-cli/builds/gh-pages/phar/wp-cli.phar
chmod +x wp-cli.phar && mv wp-cli.phar /usr/local/bin/wp

cd /var/www/html/wordpress

# Πάντα backup πριν από αναβάθμιση
sudo -u www-data wp db export /root/wp-backup.sql
tar czf /root/wp-files-backup.tgz /var/www/html/wordpress

sudo -u www-data wp core version
sudo -u www-data wp core update
sudo -u www-data wp plugin update --all
sudo -u www-data wp theme update --all
sudo -u www-data wp core verify-checksums
```

- Εκτελούμε ως `www-data` (`sudo -u`) για να μη δημιουργηθούν αρχεία που ανήκουν στον root.
- `wp core verify-checksums`: συγκρίνει κάθε αρχείο του πυρήνα με τα επίσημα checksums. Αν κάποιο αρχείο έχει πειραχτεί (π.χ. ο επιτιθέμενος έβαλε backdoor), θα φανεί εδώ.

> ⚠️ Οι πρόσφατες εκδόσεις του WordPress απαιτούν PHP 7.2.24+ (συνιστάται 8.x). Το Debian 8 έχει PHP 5.6. Άλλος ένας λόγος για τον οποίο η μετάβαση σε υποστηριζόμενη διανομή (§3.3) είναι αναγκαία.

6.2 Αποτροπή απαρίθμησης χρηστών (WPScan `-e u`)

Το WPScan βρήκε τους `michael` και `steven` με δύο βασικούς τρόπους:

1. **Author archives:** `/?author=1` κάνει ανακατεύθυνση στο `/author/michael/`.
2. **REST API:** `/wp-json/wp/v2/users` επιστρέφει λίστα χρηστών σε JSON.

Μπλοκάρουμε και τα δύο στο `.htaccess` του WordPress (πάνω από το μπλοκ `# BEGIN WordPress`):

```
<IfModule mod_rewrite.c>
RewriteEngine On
# 1. Μπλοκάρισμα ?author=N
RewriteCond %{QUERY_STRING} (^|&)author=\d+ [NC]
RewriteRule ^ - [F,L]
# 2. Μπλοκάρισμα της λίστας χρηστών στο REST API
RewriteRule ^wp-json/wp/v2/users - [F,L]
</IfModule>
```

- `RewriteCond`: η συνθήκη. Ελέγχει αν το query string περιέχει `author=` ακολουθούμενο από αριθμό.
- `RewriteRule ^ - [F,L]`: `-` σημαίνει «μην αλλάξεις το URL», `F` επιστρέφει `403 Forbidden`, `L` σταματά την επεξεργασία κανόνων.

```
a2enmod rewrite && systemctl reload apache2
```

Επιπλέον: τα ονόματα σύνδεσης (`user_login`) δεν πρέπει να είναι ίδια με τα δημόσια ονόματα εμφάνισης (`display_name`). Αν το δημόσιο όνομα είναι «Michael», το όνομα σύνδεσης ας είναι κάτι μη προβλέψιμο.

6.3 Προστασία του `wp-config.php`

Το πρόβλημα: ο `michael` (απλός χρήστης SSH) διάβασε το `wp-config.php`, γιατί είχε δικαιώματα ανάγνωσης για όλους (`644`).

Διόρθωση δικαιωμάτων:

```
cd /var/www/html/wordpress
ls -l wp-config.php                # Πριν: -rw-r--r-- (644) → διαβάζεται από όλους

chown root:www-data wp-config.php
chmod 640 wp-config.php
ls -l wp-config.php                # Μετά: -rw-r----- root www-data
```

Ψηφίο	Ποιος	Δικαίωμα	Γιατί
`6`	owner (`root`)	read + write	Μόνο ο διαχειριστής το αλλάζει
`4`	group (`www-data`)	read	Ο Apache/PHP χρειάζεται να το διαβάσει
`0`	others	τίποτα	Ο `michael` και κάθε άλλος χρήστης δεν το βλέπει

Μην χρησιμοποιείτε τον `root` της MySQL στο WordPress. Θα φτιάξουμε ξεχωριστό χρήστη με περιορισμένα δικαιώματα στην §8.3.

Επιπλέον ρυθμίσεις στο `wp-config.php`:

```
define('DISALLOW_FILE_EDIT', true);   // Απενεργοποιεί τον επεξεργαστή θεμάτων/plugins στο
dashboard
define('FORCE_SSL_ADMIN', true);      // Το wp-admin μόνο μέσω HTTPS (όταν υπάρχει πιστοποιητικό)
define('WP_DEBUG', false);            // Χωρίς μηνύματα σφάλματος που αποκαλύπτουν διαδρομές
```

Και ανανεώνουμε τα **salts** (κλειδιά που υπογράφουν τα cookies σύνδεσης), ώστε να αποσυνδεθούν όλες οι υπάρχουσες συνεδρίες, συμπεριλαμβανομένου πιθανώς του επιτιθέμενου:

```
sudo -u www-data wp config shuffle-salts
```

6.4 Δικαιώματα αρχείων όλου του WordPress

Ο κανόνας είναι: **ο web server διαβάζει, δεν γράφει** (εκτός από τον φάκελο uploads).

```
cd /var/www/html/wordpress
chown -R root:www-data .
find . -type d -exec chmod 750 {} \;
find . -type f -exec chmod 640 {} \;

# Μόνο ο φάκελος uploads χρειάζεται εγγραφή από τον web server
chown -R www-data:www-data wp-content/uploads
```

Και απαγορεύουμε την εκτέλεση PHP μέσα στα uploads, ώστε ένα ανεβασμένο αρχείο `shell.php` να μη μπορεί να τρέξει:

```
# /var/www/html/wordpress/wp-content/uploads/.htaccess
<FilesMatch "\.(php|phtml|php\d)$">
    Require all denied
</FilesMatch>
```

---

7. Θωράκιση SSH & πολιτική κωδικών

Αυτό ήταν το **κρίσιμο** σημείο εισόδου: το `hydra` βρήκε το `michael:michael`. Τρία ανεξάρτητα λάθη συνέβαλαν:

1. Το SSH δεχόταν **κωδικούς** αντί για κλειδιά.
2. Δεν υπήρχε **όριο αποτυχημένων προσπαθειών**.
3. Ο κωδικός ήταν **ίδιος με το όνομα χρήστη**.

7.1 Αυθεντικοποίηση με κλειδιά αντί για κωδικούς

Ένα κλειδί Ed25519 είναι πρακτικά αδύνατο να σπάσει με brute-force, σε αντίθεση με έναν κωδικό που μπορεί να βρίσκεται στο `rockyou.txt`.

Στον υπολογιστή του διαχειριστή (όχι στο Raven):

```
ssh-keygen -t ed25519 -C "admin@lab"     # Δημιουργεί ζεύγος ιδιωτικού/δημόσιου κλειδιού
ssh-copy-id michael@<RAVEN_IP>           # Αντιγράφει το δημόσιο κλειδί στο ~/.ssh/authorized_keys
ssh michael@<RAVEN_IP>                   # Δοκιμή: πρέπει να μπει ΧΩΡΙΣ κωδικό
```

> ⚠️ **Η αυθεντικοποίηση με κωδικό δεν πρέπει να απενεργοποιηθεί προτού επιβεβαιωθεί ότι η σύνδεση με κλειδί λειτουργεί.** Διαφορετικά υπάρχει κίνδυνος πλήρους απώλειας πρόσβασης στο σύστημα.

7.2 Ρύθμιση του `sshd_config`

```
cp /etc/ssh/sshd_config /etc/ssh/sshd_config.bak
nano /etc/ssh/sshd_config
```

```
PermitRootLogin no               # Ο root δεν συνδέεται απευθείας, μόνο μέσω su/sudo
PasswordAuthentication no        # Μόνο κλειδιά: το hydra δεν έχει τι να δοκιμάσει
PubkeyAuthentication yes
PermitEmptyPasswords no
ChallengeResponseAuthentication no
MaxAuthTries 3                   # Μέγιστο 3 προσπάθειες ανά σύνδεση
LoginGraceTime 30                # 30 δευτ. για να ολοκληρωθεί η σύνδεση
MaxStartups 10:30:60             # Περιορίζει ταυτόχρονες μη-αυθεντικοποιημένες συνδέσεις
AllowUsers michael steven        # Λευκή λίστα: μόνο αυτοί μπορούν να συνδεθούν
X11Forwarding no
ClientAliveInterval 300
ClientAliveCountMax 2            # Αποσύνδεση αδρανών συνεδριών μετά από ~10 λεπτά
```

Σημαντικές επεξηγήσεις:

- **`MaxStartups 10:30:60`**: μετά από 10 ανοιχτές μη-αυθεντικοποιημένες συνδέσεις, ο `sshd` αρχίζει να απορρίπτει νέες με πιθανότητα 30%, η οποία αυξάνεται γραμμικά έως 100% στις 60 συνδέσεις. Ο μηχανισμός αυτός επιβραδύνει δραστικά τα εργαλεία που ανοίγουν πολλές παράλληλες συνδέσεις, όπως το `hydra -t 16`.
- **`AllowUsers`**: ακόμη και αν δημιουργηθεί αργότερα κάποιος λογαριασμός με αδύναμο κωδικό (π.χ. λογαριασμός υπηρεσίας), δεν θα μπορεί να συνδεθεί μέσω SSH.

```
sshd -t                      # Έλεγχος σύνταξης. Καμία έξοδος = ΟΚ
systemctl reload ssh
```

> 💡 Κρατήστε **ανοιχτή** την τρέχουσα συνεδρία SSH και δοκιμάστε σύνδεση από **δεύτερο** τερματικό. Αν κάτι πάει στραβά, έχετε ακόμη πρόσβαση για να το διορθώσετε.

7.3 Πολιτική ισχυρών κωδικών (PAM)

Ακόμα και με κλειδιά στο SSH, οι κωδικοί χρησιμοποιούνται για `sudo`, `su` και την κονσόλα. Το **PAM** (Pluggable Authentication Modules) ελέγχει κάθε νέο κωδικό πριν τον δεχτεί.

```
apt install libpam-pwquality
```

Στο `/etc/security/pwquality.conf`:

```
minlen = 14          # Ελάχιστο μήκος 14 χαρακτήρων
minclass = 3         # Τουλάχιστον 3 κατηγορίες (πεζά, κεφαλαία, ψηφία, σύμβολα)
maxrepeat = 3        # Όχι πάνω από 3 ίδιους χαρακτήρες στη σειρά
usercheck = 1        # Απορρίπτει κωδικό που περιέχει το όνομα χρήστη (michael:michael!)
dictcheck = 1        # Απορρίπτει λέξεις λεξικού
enforce_for_root
```

Η ρύθμιση `usercheck = 1` θα είχε αποτρέψει **ακριβώς** τον κωδικό `michael`.

7.4 Κλείδωμα λογαριασμού μετά από αποτυχίες

Το PAM μπορεί να κλειδώνει προσωρινά λογαριασμό μετά από διαδοχικές αποτυχίες. Στο Debian 8 το module είναι το `pam_tally2` (στα νεότερα συστήματα το `pam_faillock`).

Debian 8 — `/etc/pam.d/common-auth` (στην αρχή του αρχείου):

```
auth required pam_tally2.so onerr=fail deny=5 unlock_time=900
```

Debian 11+ / Ubuntu 22.04+ — `/etc/security/faillock.conf`:

```
deny = 5
unlock_time = 900
fail_interval = 900
```

Μετά από 5 αποτυχίες ο λογαριασμός κλειδώνει για 15 λεπτά. Με ρυθμό 5 δοκιμές ανά 15 λεπτά, η δοκιμή των 14 εκατομμυρίων κωδικών του `rockyou.txt` θα χρειαζόταν περίπου 80 χρόνια.

```
pam_tally2 --user michael            # Πόσες αποτυχίες έχει ο χρήστης (Debian 8)
pam_tally2 --user michael --reset    # Ξεκλείδωμα από τον διαχειριστή
```

7.5 Αλλαγή των εκτεθειμένων κωδικών

Οι κωδικοί `michael` και `pink84` είναι πλέον **γνωστοί**. Αλλάζουν υποχρεωτικά:

```
passwd michael
passwd steven
passwd root
chage -d 0 michael     # Αναγκάζει τον χρήστη να αλλάξει κωδικό στην επόμενη σύνδεση
```

---

8. Θωράκιση MySQL

8.1 Τι πήγε στραβά

Από το `wp-config.php` ο επιτιθέμενος πήρε τον κωδικό του **`root` της MySQL** (`R@v3nSecurity`). Με αυτόν διάβασε **όλες** τις βάσεις, εξήγαγε hashes και (στο Raven 2) έγραψε αρχεία στον δίσκο. Το WordPress δεν χρειάζεται καθόλου αυτά τα δικαιώματα.

8.2 Βασική θωράκιση με `mysql_secure_installation`

```
mysql_secure_installation
```

Το εργαλείο κάνει διαδοχικά:

1. Αλλαγή του κωδικού του `root` της βάσης (απαντήστε **Y** και ορίστε **νέο** κωδικό).
2. Διαγραφή ανώνυμων χρηστών.
3. Απαγόρευση σύνδεσης του `root` από απομακρυσμένους υπολογιστές.
4. Διαγραφή της δοκιμαστικής βάσης `test` .
5. Επαναφόρτωση των πινάκων δικαιωμάτων.

8.3 Ξεχωριστός χρήστης για την εφαρμογή (Least Privilege)

```
-- Συνδεόμαστε ως root της βάσης: mysql -u root -p
CREATE USER 'wp_app'@'localhost' IDENTIFIED BY 'Ένας-Μακρύς-Τυχαίος-Κωδικός-2026!';

GRANT SELECT, INSERT, UPDATE, DELETE, CREATE, ALTER, INDEX, DROP
  ON wordpress.* TO 'wp_app'@'localhost';

FLUSH PRIVILEGES;
SHOW GRANTS FOR 'wp_app'@'localhost';
```

- `'wp_app'@'localhost'` : ο χρήστης μπορεί να συνδεθεί **μόνο** από τον ίδιο server.
- `ON wordpress.*` : δικαιώματα **μόνο** στη βάση `wordpress` , όχι στη `mysql` (εκεί που βρίσκονται οι κωδικοί των χρηστών της βάσης).
- **Δεν** δίνουμε `FILE` (ανάγνωση/εγγραφή αρχείων στον δίσκο), `SUPER` , `GRANT OPTION` , `PROCESS` .

Ενημερώνουμε το `wp-config.php` :

```
define('DB_USER', 'wp_app');
define('DB_PASSWORD', 'Ένας-Μακρύς-Τυχαίος-Κωδικός-2026!');
```

Ακόμα κι αν ο επιτιθέμενος διαβάσει ξανά το `wp-config.php`, θα έχει πρόσβαση μόνο στη βάση του WordPress, όχι σε όλο τον server βάσεων.

8.4 Ρυθμίσεις του `mysqld`

Στο `/etc/mysql/my.cnf` (ή `/etc/mysql/mysql.conf.d/mysqld.cnf`), ενότητα `[mysqld]`:

```
[mysqld]
user            = mysql      # Η υπηρεσία τρέχει ως χρήστης mysql, ΟΧΙ ως root
bind-address    = 127.0.0.1  # Ακούει μόνο τοπικά, δεν είναι προσβάσιμη από το δίκτυο
local-infile    = 0          # Απενεργοποιεί το LOAD DATA LOCAL INFILE
secure-file-priv = /var/lib/mysql-files   # Περιορίζει LOAD_FILE / INTO OUTFILE σε έναν φάκελο
symbolic-links   = 0
```

```
mkdir -p /var/lib/mysql-files && chown mysql:mysql /var/lib/mysql-files && chmod 750
/var/lib/mysql-files
systemctl restart mysql

# Επαλήθευση: ποιος χρήστης τρέχει τη MySQL;
ps -o user,cmd -C mysqld
```

Το **`user = mysql`** είναι κρίσιμο: στο Raven η MySQL έτρεχε ως `root`, οπότε ό,τι έκανε η βάση (π.χ. εγγραφή αρχείου) γινόταν με δικαιώματα root. Η εκμετάλλευση αυτού του λάθους αναλύεται στο Hardening-Guide του Raven 2.

---

9. Διόρθωση sudo

9.1 Γιατί το `NOPASSWD: /usr/bin/python` ισοδυναμεί με root

```
User steven may run the following commands on raven:
    (ALL) NOPASSWD: /usr/bin/python
```

Ο διαχειριστής ίσως σκέφτηκε «δίνω στον steven μόνο την Python». Όμως η Python είναι **πλήρης γλώσσα προγραμματισμού**: μπορεί να ανοίξει shell, να διαβάσει/γράψει οποιοδήποτε αρχείο, να αλλάξει κωδικούς. Το ίδιο ισχύει για `perl`, `ruby`, `vim`, `less`, `find`, `awk`, `nmap`, `tar` κ.ά. Ο ιστότοπος **GTFOBins** καταγράφει εκατοντάδες τέτοια προγράμματα.

> 📌 **Κανόνας:** δώσε sudo σε ένα πρόγραμμα **μόνο** αν είσαι βέβαιος ότι δεν μπορεί να εκτελέσει άλλες εντολές ή να γράψει αυθαίρετα αρχεία.

9.2 Έλεγχος

```
sudo -l -U steven              # Τι μπορεί να τρέξει ο steven
grep -rn NOPASSWD /etc/sudoers /etc/sudoers.d/
```

9.3 Διόρθωση

Πάντα με `visudo`, ποτέ με απευθείας επεξεργασία του αρχείου:

```
visudo                      # Για το /etc/sudoers
visudo -f /etc/sudoers.d/steven
```

Το `visudo` ελέγχει τη σύνταξη πριν αποθηκεύσει. Ένα συντακτικό λάθος στο `sudoers` μπορεί να απενεργοποιήσει το `sudo` εντελώς.

Αφαιρούμε τη γραμμή:

```
steven ALL=(ALL) NOPASSWD: /usr/bin/python
```

Αν ο `steven` χρειάζεται πράγματι να κάνει κάτι συγκεκριμένο ως root (π.χ. να επανεκκινεί τον Apache), του δίνουμε **ακριβώς** αυτό:

```
steven ALL=(root) /bin/systemctl reload apache2
```

- Χωρίς `NOPASSWD`: ο χρήστης πρέπει να δώσει τον κωδικό του. Έτσι, αν κάποιος κλέψει τη συνεδρία του, δεν παίρνει αυτόματα δικαιώματα.
- Συγκεκριμένη εντολή **με** παραμέτρους: δεν μπορεί να τρέξει `systemctl` με άλλες παραμέτρους.

Αν χρειάζεται να τρέξει **δικό του** script ως root, το script πρέπει να ανήκει στον root και να μην είναι εγγράψιμο από τον `steven`:

```
chown root:root /usr/local/sbin/backup.sh
chmod 755 /usr/local/sbin/backup.sh     # Ο steven εκτελεί, ΔΕΝ τροποποιεί
```

9.4 Ενισχυτικές ρυθμίσεις sudo

```
Defaults    use_pty                         # Το sudo τρέχει σε δικό του pseudo-terminal
Defaults    logfile="/var/log/sudo.log"     # Καταγραφή κάθε χρήσης του sudo
Defaults    timestamp_timeout=5             # Ο κωδικός «θυμάται» μόνο για 5 λεπτά
Defaults    passwd_tries=3
```

---

10. Hashes κωδικών & επαναχρησιμοποίηση

10.1 Τι έδειξε το John the Ripper

Ο John έσπασε το hash του `steven` (`pink84`) επειδή ο κωδικός υπήρχε στο `rockyou.txt`. **Το πρόβλημα δεν ήταν ο αλγόριθμος, ήταν ο κωδικός.** Κανένας αλγόριθμος hashing δεν προστατεύει έναν κωδικό που βρίσκεται σε δημόσιο λεξικό: ο επιτιθέμενος απλώς υπολογίζει το hash κάθε λέξης και συγκρίνει.

Το δεύτερο πρόβλημα ήταν η **επαναχρησιμοποίηση**: ο ίδιος κωδικός ίσχυε στο WordPress **και** στο λειτουργικό σύστημα.

10.2 Διόρθωση

1. **Μοναδικοί κωδικοί ανά σύστημα.** Ο κωδικός WordPress ≠ κωδικός Linux ≠ κωδικός MySQL ≠ κωδικός root. Χρησιμοποιήστε password manager (π.χ. KeePassXC).
2. **Ισχυρότεροι αλγόριθμοι.** Το WordPress από την έκδοση 6.8 αποθηκεύει τους κωδικούς με **bcrypt** αντί για phpass, οπότε η αναβάθμιση (§6.1) βοηθά και εδώ. Οι υπάρχοντες κωδικοί μετατρέπονται αυτόματα στην επόμενη σύνδεση του χρήστη.
3. **Ισχυρό hashing στο Linux.** Ελέγξτε τον αλγόριθμο στο `/etc/shadow`:

```
bash awk -F: '$2 ~ /^\$/ {print $1, substr($2,1,4)}' /etc/shadow
```

Πρόθεμα	Αλγόριθμος	Εκτίμηση
`$1$`	MD5-crypt	Ακατάλληλο
`$6$`	SHA-512-crypt	Αποδεκτό
`$y$`	yescrypt	Προεπιλογή σε Debian 11+, το καλύτερο

Στο `/etc/login.defs` ορίζουμε `ENCRYPT_METHOD SHA512` (ή `YESCRYPT` σε νεότερα συστήματα). Ισχύει για **νέους** κωδικούς, γι' αυτό η αλλαγή κωδικών της §7.5 γίνεται **μετά** από αυτή τη ρύθμιση.

4. **Ξεχωριστός κωδικός root, κλειδωμένος root.** Αφού οι διαχειριστές χρησιμοποιούν `sudo`, ο άμεσος λογαριασμός root μπορεί να κλειδωθεί:

```
bash passwd -l root # Κλειδώνει τον κωδικό του root. Το `su -` δεν δουλεύει πλέον
```

> ⚠️ Κάντε το **μόνο** αφού επιβεβαιώσετε ότι τουλάχιστον ένας χρήστης έχει πλήρη δικαιώματα sudo, αλλιώς χάνετε πρόσβαση διαχειριστή.

---

11. Επαλήθευση

Μια διόρθωση δεν θεωρείται ολοκληρωμένη μέχρι να **αποδειχτεί** ότι η επίθεση αποτυγχάνει. Από το Kali, επαναλαμβάνουμε κάθε βήμα:

```
# 1. Σάρωση: πρέπει να φαίνεται μόνο το 80 (και το 22 μόνο από το δίκτυο διαχείρισης)
nmap -sV -p- <RAVEN_IP>

# 2. Directory listing: πρέπει 403
curl -I http://<RAVEN_IP>/vendor/

# 3. Απαρίθμηση χρηστών: πρέπει 403
curl -I "http://<RAVEN_IP>/wordpress/?author=1"
curl -I http://<RAVEN_IP>/wordpress/wp-json/wp/v2/users

# 4. SSH με κωδικό: πρέπει "Permission denied (publickey)"
ssh -o PreferredAuthentications=password michael@<RAVEN_IP>

# 5. Hydra: πρέπει να αποτύχει αμέσως
hydra -l michael -P /usr/share/wordlists/rockyou.txt ssh://<RAVEN_IP>
```

Στο **Raven**:

```
# 6. Ο michael δεν διαβάζει το wp-config
sudo -u michael cat /var/www/html/wordpress/wp-config.php     # → Permission denied

# 7. Ο steven δεν έχει πλέον sudo python
sudo -l -U steven

# 8. Η MySQL τρέχει ως mysql και ακούει μόνο τοπικά
ps -o user,cmd -C mysqld
ss -tlnp | grep 3306                                          # → 127.0.0.1:3306
```

Καταγράψτε τα αποτελέσματα σε πίνακα «Πριν / Μετά». Αυτός ο πίνακας είναι το βασικό παραδοτέο μιας αναφοράς αποκατάστασης.

---

12. Λίστα ελέγχου & ερωτήσεις αυτοελέγχου

12.1 Λίστα ελέγχου

- [ ] Snapshot πριν τις αλλαγές
- [ ] Ενημέρωση πακέτων / σχέδιο μετάβασης σε υποστηριζόμενο Debian
- [ ] `rpcbind` απενεργοποιημένο, firewall `default deny`, SSH μόνο από δίκτυο διαχείρισης
- [ ] Σχόλια HTML καθαρισμένα, `Options -Indexes`, `/vendor` μπλοκαρισμένο
- [ ] `ServerTokens Prod`, `ServerSignature Off`, `expose_php Off`
- [ ] WordPress ενημερωμένο, απαρίθμηση χρηστών μπλοκαρισμένη, `DISALLOW_FILE_EDIT`
- [ ] `wp-config.php` σε `640 root:www-data`, νέα salts

- [ ] SSH: κλειδιά, `PasswordAuthentication no`, `PermitRootLogin no`, `MaxAuthTries 3`, `AllowUsers`
- [ ] PAM: `pwquality` (`usercheck=1`) και κλείδωμα μετά από 5 αποτυχίες
- [ ] Όλοι οι εκτεθειμένοι κωδικοί άλλαξαν
- [ ] MySQL: νέος κωδικός root, χρήστης `wp_app` με ελάχιστα δικαιώματα, `bind-address 127.0.0.1`, `user = mysql`
- [ ] `sudoers`: αφαιρέθηκε το `NOPASSWD: /usr/bin/python`
- [ ] Επαλήθευση όλων των βημάτων της §11

12.2 Ερωτήσεις αυτοελέγχου

1. Γιατί η απόκρυψη της έκδοσης του Apache (`ServerTokens Prod`) **δεν** θεωρείται διόρθωση ευπάθειας; Τι προσφέρει τελικά;
2. Ο `michael` είχε κωδικό ίδιο με το όνομα χρήστη. Ποια **τρία** ανεξάρτητα μέτρα αυτού του εγχειριδίου θα είχαν αποτρέψει, το καθένα μόνο του, την επιτυχία του `hydra`;
3. Γιατί το `wp-config.php` πρέπει να έχει group `www-data` και όχι να ανήκει εξ ολοκλήρου στον `www-data`;
4. Εξηγήστε γιατί το `sudo` σε `/usr/bin/python` ισοδυναμεί με πλήρη πρόσβαση root. Αναφέρετε τρία ακόμη προγράμματα με την ίδια ιδιότητα.
5. Ποια είναι η διαφορά μεταξύ `systemctl disable` και `systemctl mask`;
6. Αν ο κωδικός του `steven` είχε αποθηκευτεί με bcrypt αντί για phpass, θα είχε αποτύχει το John the Ripper; Αιτιολογήστε.
7. Ποιο δικαίωμα MySQL επιτρέπει τις εντολές `LOAD_FILE()` και `INTO DUMPFILE`, και γιατί δεν πρέπει να το έχει ο χρήστης της εφαρμογής;

Ενδεικτικές απαντήσεις

1. Κρύβει μόνο την **πληροφορία έκδοσης** στα HTTP headers, δεν διορθώνει κανένα bug — ο server παραμένει εξίσου ευάλωτος αν τρέχει παλιά έκδοση. Προσφέρει μείωση του **information disclosure** (δυσκολεύει την αντιστοίχιση έκδοση→γνωστό CVE), δηλαδή είναι μέτρο μείωσης θορύβου, όχι remediation. Η πραγματική διόρθωση είναι η ενημέρωση.
2. (1) **Password policy** (PAM `pwquality` με `usercheck=1`) που απορρίπτει κωδικό ίδιο με το username. (2) **Account lockout** (pam_faillock/tally, κλείδωμα μετά από 5 αποτυχίες) που σταματά το brute-force. (3) `PasswordAuthentication no` **στο SSH** (μόνο κλειδιά) — το hydra δεν έχει πια κωδικό να μαντέψει. Κάθε ένα από μόνο του αρκεί.
3. Το αρχείο πρέπει να το **διαβάζει** ο web server αλλά όχι να το **τροποποιεί**. Με owner `root`, group `www-data` και δικαιώματα `640`, ο www-data έχει μόνο ανάγνωση. Αν ανήκε εξ ολοκλήρου στον www-data, μια παραβίαση της εφαρμογής (RCE ως www-data) θα επέτρεπε να **ξαναγραφτεί** το config (κακόβουλος κώδικας, αλλαγή credentials). Owner root = η παραβιασμένη διεργασία δεν μπορεί να το πειράξει.
4. Το `sudo` τρέχει την Python με EUID 0, και η Python είναι **πλήρης διερμηνέας** που εκτελεί αυθαίρετο κώδικα — άρα `os.system("/bin/bash")` δίνει root shell, δικαίωμα σε interpreter = δικαίωμα σε **τα πάντα**. Άλλα παραδείγματα (GTFOBins): `perl`, `ruby`, `awk` (επίσης `find`, `vim`, `less`, `nmap`).

5. Το `disable` αφαιρεί μόνο την **αυτόματη εκκίνηση στο boot**, η υπηρεσία μπορεί ακόμη να ξεκινήσει χειροκίνητα ή ως εξάρτηση. Το `mask` τη συνδέει με το `/dev/null` ώστε να **μην μπορεί να ξεκινήσει με κανέναν τρόπο** μέχρι `unmask`. Για υπηρεσίες που θέλουμε σίγουρα ανενεργές (π.χ. `rpcbind`) χρησιμοποιούμε `mask`.
6. Όχι εγγυημένα — το John υποστηρίζει bcrypt. Η διαφορά είναι η **ταχύτητα**: ο bcrypt με υψηλό cost factor είναι σκόπιμα πολύ πιο αργός ανά δοκιμή, οπότε οι δοκιμές/δευτερόλεπτο πέφτουν κατακόρυφα. Ένας αδύναμος κωδικός (`pink84`, που υπάρχει στο rockyou) θα έσπαγε ούτως ή άλλως, ένας ισχυρός κωδικός με bcrypt θα γινόταν πρακτικά ανέφικτος. Ο bcrypt αυξάνει το κόστος, δεν κάνει αδύνατο το cracking αδύναμων κωδικών.
7. Το προνόμιο `FILE`. Επιτρέπει ανάγνωση (`LOAD_FILE()`) και εγγραφή (`INTO OUTFILE/DUMPFILE`) αρχείων με τα δικαιώματα της διεργασίας mysqld — βάση για UDF/webshell επιθέσεις. Ο χρήστης της εφαρμογής χρειάζεται μόνο SELECT/INSERT/UPDATE/DELETE στη δική του βάση, το `FILE` είναι global και επικίνδυνο (least privilege), και επιπλέον το περιορίζουμε με `secure_file_priv`.

Πηγές

- Debian Security Manual — Securing Debian
- WordPress — Hardening WordPress
- OpenSSH — sshd_config(5)
- MySQL — Security Guidelines
- GTFOBins — python
- CIS Benchmarks — Debian Linux / Apache HTTP Server